

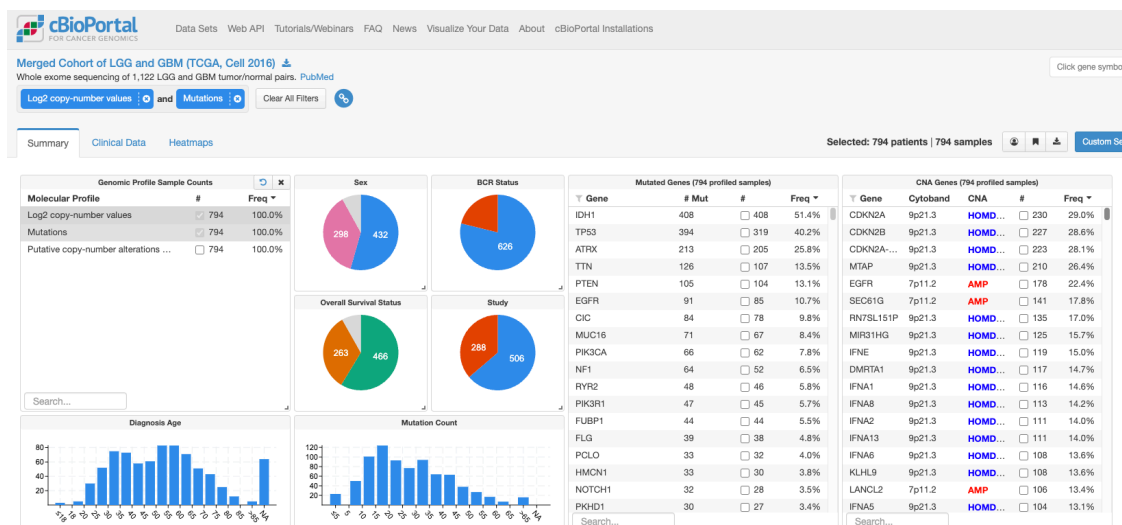
day2_hw_lgg_gbm

March 14, 2025

```
[1]: import os
import flexynesis
import torch
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import random
import lightning as pl
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
```

Seed set to 42

1 Finding Survival Markers in Lower Grade Glioma (LGG) and Glioblastoma Multiforme (GBM)seed



Here, we demonstrate the capabilities of `flexynesis` on a multi-omic dataset of 506 Brain Lower Grade Glioma (LGG) and 288 Glioblastoma Multiforme (GBM) samples with matching mutation and copy number alteration data downloaded from the [cbioportal](https://www.cbioportal.org/). The data was split into `train`

(70% of the samples) and `test` (30% of the samples) data folders. The data files were processed to follow the same nomenclature.

- `cna.csv` contains “copy number alteration” data
- `mut.csv` contains “mutation” data, which is a binary matrix of genes versus samples.
- `clin.csv` contains “clinical/sample metadata”, which is a table of clinical parameters such as age, sex, disease type, histological diagnosis, and overall survival time and status.

1.1 Data Download

The data can be downloaded as follows:

```
[2]: if not os.path.exists("lgggbm_tcga_pub_processed"):
      !wget -O lgggbm_tcga_pub_processed.tgz "https://bimsbstatic.mdc-berlin.de/
      ↪akalin/buyar/flexynesis-benchmark-datasets/lgggbm_tcga_pub_processed.tgz" &&
      ↪tar -xzf lgggbm_tcga_pub_processed.tgz
```

```
--2025-03-14 20:09:02-- https://bimsbstatic.mdc-berlin.de/akalin/buyar/flexynesis-benchmark-datasets/lgggbm_tcga_pub_processed.tgz
Resolving bimsbstatic.mdc-berlin.de (bimsbstatic.mdc-berlin.de)...
141.80.181.46, 141.80.181.47
connected. to bimsbstatic.mdc-berlin.de (bimsbstatic.mdc-berlin.de)|141.80.181.46|:443...
HTTP request sent, awaiting response... 200 OK
Length: 1817945 (1.7M) [application/octet-stream]
Saving to: 'lgggbm_tcga_pub_processed.tgz'
```

```
lgggbm_tcga_pub_pro 100%[=====>] 1.73M --.-KB/s in 0.1s
```

```
2025-03-14 20:09:02 (18.0 MB/s) - 'lgggbm_tcga_pub_processed.tgz' saved
[1817945/1817945]
```

```
lgggbm_tcga_pub_processed/
lgggbm_tcga_pub_processed/train/
lgggbm_tcga_pub_processed/train/mut.csv
lgggbm_tcga_pub_processed/train/cna.csv
lgggbm_tcga_pub_processed/train/clin.csv
lgggbm_tcga_pub_processed/test/
lgggbm_tcga_pub_processed/test/mut.csv
lgggbm_tcga_pub_processed/test/cna.csv
lgggbm_tcga_pub_processed/test/clin.csv
```

1.2 Importing Train and Test Datasets

We import train and test datasets including mutations and copy number alterations. We rank genes by Laplacian Scores and pick top 10% of the genes, while removing highly redundant genes with a correlation score threshold of 0.8 and a variance threshold of 50%. By setting `concatenate` to `False`, we will be doing an `intermediate` fusion of omic layers.

```
[3]: data_importer = flexynesis.DataImporter(path='lgggbm_tcga_pub_processed',
                                             data_types = ['mut', 'cna'],
                                             log_transform=False,
                                             concatenate=False, top_percentile=10,
                                             min_features=1000, correlation_threshold=0.8,
                                             variance_threshold=0.5)
train_dataset, test_dataset = data_importer.import_data()
```

[INFO] ===== Importing Data =====

[INFO] Validating data folders...

[INFO] ----- Reading Data -----

[INFO] Importing lgggbm_tcga_pub_processed/train/clin.csv...

[INFO] Importing lgggbm_tcga_pub_processed/train/cna.csv...

[INFO] Importing lgggbm_tcga_pub_processed/train/mut.csv...

[INFO] ----- Reading Data -----

[INFO] Importing lgggbm_tcga_pub_processed/test/clin.csv...

[INFO] Importing lgggbm_tcga_pub_processed/test/cna.csv...

[INFO] Importing lgggbm_tcga_pub_processed/test/mut.csv...

[INFO] ----- Checking for problems with the input data -----

[INFO] Data structure is valid with no errors or warnings.

[INFO] ----- Processing Data (train) -----

[INFO] ----- Cleaning Up Data -----

[INFO] working on layer: mut

[INFO] Number of NA values: 0

[INFO] DataFrame mut - Removed 5561 features.

[INFO] working on layer: cna

[INFO] Number of NA values: 0

[INFO] DataFrame cna - Removed 12375 features.

[INFO] DataFrame mut - Removed 0 samples (0.00%).

[INFO] DataFrame cna - Removed 0 samples (0.00%).

[INFO] Implementing feature selection using laplacian score for layer: mut with 5503 features and 556 samples

Calculating Laplacian scores: 100% | 5503/5503 [00:00<00:00, 11225.40it/s]

Filtering redundant features: 100% | 1000/1000 [00:00<00:00, 14500.82it/s]

[INFO] Implementing feature selection using laplacian score for layer: cna with 12371 features and 556 samples

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```

OpenBLAS Warning : Detect OpenMP Loop and this application may hang. Please
rebuild the library with USE_OPENMP=1 option.
OpenBLAS Warning : Detect OpenMP Loop and this application may hang. Please
rebuild the library with USE_OPENMP=1 option.
OpenBLAS Warning : Detect OpenMP Loop and this application may hang. Please
rebuild the library with USE_OPENMP=1 option.
OpenBLAS Warning : Detect OpenMP Loop and this application may hang. Please
rebuild the library with USE_OPENMP=1 option.
Calculating Laplacian scores: 100%|      | 12371/12371 [00:01<00:00,
11838.28it/s]
Filtering redundant features: 100%|      | 1237/1237 [00:00<00:00,
237181.90it/s]

```

```
[INFO] ----- Processing Data (test) -----
```

```
[INFO] ----- Cleaning Up Data -----
```

```

[INFO] working on layer:  mut
[INFO] Number of NA values:  0
[INFO] DataFrame mut - Removed 5627 features.

```

```

[INFO] working on layer:  cna
[INFO] Number of NA values:  0
[INFO] DataFrame cna - Removed 12382 features.
[INFO] DataFrame mut - Removed 0 samples (0.00%).
[INFO] DataFrame cna - Removed 0 samples (0.00%).

```

```
[INFO] ----- Harmonizing Data Sets -----
```

```
[INFO] ----- Finished Harmonizing -----
```

```
[INFO] ----- Normalizing Data -----
```

```

[INFO] ----- Normalizing Data -----
[INFO] Training Data Stats:  {'feature_count in: cna': 1237, 'feature_count in:
mut': 321, 'sample_count': 556}
[INFO] Test Data Stats:    {'feature_count in: cna': 1237, 'feature_count in:
mut': 321, 'sample_count': 238}
[INFO] Merging Feature Logs...
[INFO] Data import successful.

```

1.3 1. Exploratory Data Analysis

Before building any machine learning models on the data, it is important to first familiarize yourself with the data you are working with. It is important to know the available data matrices, their sizes/shapes, available clinical variables and how they are distributed.

Below you are asked to do simple explorations of the available data.

1.4 1.1 Print the shapes of the available data matrices

- How many features and samples are available per data type in train/test datasets?

```
[4]: !wc -l ./lgggbm_tcga_pub_processed/train/*

557 ./lgggbm_tcga_pub_processed/train/clin.csv
24747 ./lgggbm_tcga_pub_processed/train/cna.csv
11065 ./lgggbm_tcga_pub_processed/train/mut.csv
36369 total
```

```
[5]: !wc -l ./lgggbm_tcga_pub_processed/test/*

239 ./lgggbm_tcga_pub_processed/test/clin.csv
24747 ./lgggbm_tcga_pub_processed/test/cna.csv
11065 ./lgggbm_tcga_pub_processed/test/mut.csv
36051 total
```

1.5 1.2 Explore sample annotations

- What are the available clinical variables? Are they available in both train and test datasets? (See .ann)

```
[53]: print(dir(train_dataset))

['__add__', '__annotations__', '__class__', '__class_getitem__', '__delattr__',
 '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__getattr__', '__getitem__', '__gt__', '__hash__', '__init__',
 '__init_subclass__', '__le__', '__len__', '__lt__', '__module__', '__ne__',
 '__new__', '__orig_bases__', '__parameters__', '__reduce__', '__reduce_ex__',
 '__repr__', '__setattr__', '__sizeof__', '__slots__', '__str__',
 '__subclasshook__', '__weakref__', '_is_protocol', 'ann', 'dat', 'feature_ann',
 'features', 'get_dataset_stats', 'get_feature_subset', 'label_mappings',
 'samples', 'subset', 'variable_types']
```

```
[52]: train_dataset.ann
```

```
[52]: {'AGE': tensor([44., 39., 45., 45., 59., 39., 52., 30., 58., 31., 28., 38., 61.,
61.,
78., 51., 35., 30., 31., 27., 50., 32., 46., 24., 48., 55., 25., 30.,
77., 79., 81., 66., 26., 42., 48., 68., 67., 59., 74., 64., 78., 40.,
47., 70., 78., 40., 51., 58., nan, 65., 30., 70., 28., 41., 57., 65.,
36., 36., 40., 63., 55., 62., 38., 56., 43., 37., 54., 66., 18., 50.,
68., 21., 42., 44., 42., 50., 40., 20., 74., 72., 74., 21., 56., 77.,
41., 54., nan, 60., 57., 43., 39., 33., 75., 62., 37., 60., 55., 30.,
49., 58., 21., 21., 32., nan, 53., 73., 52., 52., 68., 47., 76., 54.,
72., 46., 28., 71., 65., 63., 58., 35., 32., 34., nan, 63., 53., 34.,
30., 43., 70., 43., 69., 52., 62., 74., 52., 67., 39., 52., 31., 54.,
33., nan, 60., 33., 42., 60., 39., 63., nan, 62., 54., 54., 59., 26.,
52., 30., 37., 48., 66., 52., 44., 41., 56., 44., 57., 25., nan, 49.,
```

```

66., 34., 51., 49., 63., 76., 72., 65., 31., 60., 33., 37., 43., 33.,
49., 59., 58., 36., 69., 72., 29., 62., 60., 51., 60., 64., 35., 78.,
76., 47., 67., 40., 33., 61., 82., 72., 59., 65., 27., nan, 53., 37.,
31., 60., 39., 49., 78., 68., 29., 29., 50., 59., 43., 64., 81., nan,
45., 29., 27., 57., 23., 51., 58., 38., 58., 45., 75., 53., 63., 31.,
31., 53., 33., 28., 76., 51., 48., 44., 46., 77., 68., 29., 54., 67.,
65., 53., 47., 40., 17., 48., 72., 48., 81., 54., 59., 75., 51., nan,
70., 86., 26., 30., nan, 63., 70., 61., 54., 35., 62., 47., 65., 30.,
53., 25., 47., 41., nan, 64., 62., 48., 50., 62., nan, 53., 38., 53.,
38., 67., 53., 24., 51., 55., 66., 74., 60., 67., nan, nan, 65., 51.,
75., 51., 81., nan, 43., 62., 36., 47., 51., 60., nan, 23., 72., 50.,
71., 45., 76., 57., 37., 66., 34., 61., 26., 22., 35., 24., nan, 68.,
nan, 49., 61., 30., nan, 32., 52., 74., 47., 76., 50., 33., 76., 36.,
38., 29., nan, 40., 73., 57., 32., 60., 43., 53., nan, 46., 26., 36.,
39., 26., 74., 74., 57., 40., 49., 36., 33., 55., 25., 33., 31., 30.,
33., 53., 63., 39., 56., 35., nan, 54., 23., 56., 70., 25., 38., 50.,
59., 39., 65., 67., 27., 53., 74., 25., 52., 64., nan, 61., 36., 46.,
58., 40., 47., nan, 32., 60., 44., 41., 55., 69., 69., 44., nan, 72.,
27., 64., 43., nan, 28., 48., 85., 67., 46., 64., 43., 70., 29., 42.,
nan, 42., 71., 30., 32., 30., 66., 47., 58., 64., 48., nan, 14., 59.,
nan, 66., 53., 57., 72., 21., 84., 52., 44., 56., 63., 58., 31., 69.,
67., nan, 34., 66., 67., 34., 41., 32., 56., 41., 59., 38., 37., 31.,
54., 76., 40., 67., 70., 37., 49., 54., 46., 75., 63., 62., 67., 57.,
25., 20., 29., 59., nan, 88., nan, 43., 35., 34., nan, 47., 38., 39.,
58., 53., 32., 38., 29., 60., 51., 37., 40., 49., 52., 47., 51., 36.,
73., 63., nan, 33., 22., 38., 42., 45., 48., 26., 56., 34., 31., 85.,
60., 54., 81., 73., 34., 64., 45., 74., 81., 21., 60., 30., 40., 69.,
80., 56., 34., 38., 60., 69., 55., 49., 43., 65.],
dtype=torch.float64),
'OS_MONTHS': tensor([2.4200e+01, 2.3600e+01, 7.5000e+01, 1.0300e+01,
1.9600e+01, 1.7800e+01,
9.4000e+01, 2.5300e+01, 1.4900e+01, 2.2500e+01, 8.7000e+01, 2.2000e+00,
9.2000e+00, 2.9900e+01, 1.5300e+01, 1.2600e+01, 1.6430e+02, 1.5500e+01,
7.0500e+01, 3.8200e+01, 3.3100e+01, 7.2900e+01, 1.0900e+01, 1.0900e+01,
6.0300e+01, 5.0000e-01, 1.0510e+02, 5.0000e+00, 9.8000e+00, 2.0900e+01,
2.2000e+00, 3.7000e+00, 2.0000e-01, 6.1000e+00, 9.5500e+01, 5.9000e+00,
2.0400e+01, 1.8800e+01, 7.0000e+00, 1.6000e+00, 1.3000e+00, 1.4300e+01,
1.1800e+01, 9.2000e+00, 1.2000e+00, 4.4400e+01, 3.0000e+00, 5.4000e+00,
nan, 1.4700e+01, 6.7000e+00, 5.0000e+00, 1.5300e+01, 2.8000e+00,
8.0000e+00, 1.2800e+01, 4.0000e+00, 4.7900e+01, 1.1900e+01, 1.2200e+01,
2.0000e-01, 5.7900e+01, 1.3000e+01, 4.2300e+01, 3.0000e+00, 5.3000e+00,
6.5700e+01, 2.2700e+01, 6.0000e+01, 2.2900e+01, 4.5000e+00, 4.7000e+00,
4.0000e+00, 2.0000e-01, 1.5100e+01, 1.0000e-01, 3.4000e+00, 6.7000e+00,
9.4000e+00, 4.5000e+00, 2.8900e+01, 2.4000e+00, 5.4000e+00, 5.3000e+00,
1.4900e+01, 1.0000e-01, nan, 1.6900e+01, 5.1000e+00, 0.0000e+00,
2.0000e-01, 3.0100e+01, 1.2500e+01, 1.0400e+01, 1.0000e-01, 5.4000e+00,
5.7000e+00, 5.4000e+00, 2.0100e+01, 1.7800e+01, 9.4000e+00, 2.9800e+01,

```

4.6600e+01, nan, 2.1300e+01, 1.8000e+01, 1.0370e+02, 7.5000e+00,
 3.3000e+00, 6.3000e+00, 7.6000e+00, 2.6700e+01, 3.5000e+00, 1.7400e+01,
 7.8000e+00, 2.8500e+01, 4.4000e+00, 2.3700e+01, 1.7700e+01, 2.0700e+01,
 7.7800e+01, 2.1700e+01, nan, 2.0000e-01, 2.6000e+00, 7.0000e+00,
 1.4400e+01, 4.8000e+00, 1.8700e+01, 1.9000e+01, 1.5500e+01, 2.4700e+01,
 2.5000e+00, 5.5000e+00, 7.0000e+00, 3.4000e+00, 2.3000e+00, 0.0000e+00,
 4.9000e+01, 2.0000e-01, 7.5100e+01, nan, 1.1600e+01, 8.1900e+01,
 2.0800e+01, 1.1500e+01, 3.6800e+01, 1.8400e+01, nan, 1.0600e+01,
 4.7700e+01, 2.8000e+00, 4.4500e+01, 6.5000e+00, 1.0600e+01, 1.0300e+01,
 3.0100e+01, 1.5600e+01, 1.9500e+01, 1.0800e+01, 7.1000e+00, 3.2300e+01,
 4.0500e+01, 9.7000e+00, 1.1400e+01, 1.0000e-01, nan, 6.8000e+00,
 5.2100e+01, 6.6000e+00, 1.0500e+01, 7.8000e+00, 4.7600e+01, 4.8000e+00,
 1.4900e+01, 7.0000e-01, 2.0600e+01, 8.5000e+00, 1.7700e+01, 1.8900e+01,
 1.7500e+01, 1.0000e-01, 2.9600e+01, 1.4900e+01, 6.2300e+01, 1.4700e+01,
 8.0000e+00, 8.0000e+00, 1.4500e+01, 1.8900e+01, 1.4900e+01, 1.9900e+01,
 1.5900e+01, 8.3000e+00, 0.0000e+00, 6.7000e+00, 4.1000e+00, 1.8700e+01,
 7.0000e-01, 5.0000e+00, 4.4000e+01, 2.2000e+01, 8.9000e+00, 6.3000e+00,
 1.8800e+01, 5.3000e+00, 8.3700e+01, nan, 1.5400e+01, 8.6000e+00,
 6.0000e+00, 2.1000e+01, 5.9100e+01, 1.3000e+01, 4.5000e+00, 1.2700e+01,
 2.0700e+01, 7.6000e+00, 5.8000e+00, 8.9000e+00, 3.1300e+01, 5.2000e+00,
 2.7000e+00, nan, 4.6000e+01, 4.2700e+01, 6.0000e-01, 8.4000e+00,
 4.0900e+01, 2.0000e-01, 6.0000e+00, 2.9000e+00, 1.5000e+00, 1.4300e+01,
 9.5000e+00, 9.3000e+00, 5.3000e+00, 8.7400e+01, 1.7900e+01, 9.4000e+00,
 1.0500e+01, 3.6800e+01, 3.6000e+00, 4.3000e+00, 5.9300e+01, 4.3900e+01,
 9.4000e+00, 4.8000e+00, 1.0500e+01, 4.4600e+01, 1.3900e+01, 8.8000e+00,
 1.3800e+01, 1.2000e+00, 1.5700e+01, 1.6800e+01, 5.0100e+01, 6.2100e+01,
 8.5000e+00, 1.0400e+01, 2.8000e+00, 1.4700e+01, 2.0000e+00, 2.7000e+00,
 8.3000e+00, nan, 3.0200e+01, 2.0000e-01, 2.7600e+01, 5.3000e+00,
 nan, 5.2000e+00, 4.9000e+00, 9.0000e+00, 1.2900e+01, 1.3700e+01,
 1.5900e+01, 1.4300e+01, 1.3600e+01, 1.1500e+01, 1.6200e+01, 2.0000e+00,
 4.5000e+01, 4.2000e+00, nan, 8.3000e+00, 3.9500e+01, 5.0000e+00,
 4.7000e+00, 1.1000e+01, nan, 4.8000e+00, 4.6000e+00, 1.0000e-01,
 6.7400e+01, 2.4200e+01, 6.2000e+01, 1.0000e-01, 1.2200e+01, 1.4200e+01,
 1.6600e+01, 2.6600e+01, 2.4200e+01, 1.1000e+01, nan, nan,
 1.4900e+01, 2.1400e+01, 2.0000e-01, 1.2400e+01, 1.4400e+01, nan,
 9.0000e-01, 2.4200e+01, 2.1600e+01, 6.6000e+00, 1.4300e+01, 4.2000e+00,
 nan, 1.1400e+02, 7.4000e+00, 2.3000e+01, 1.2300e+01, 5.3000e+00,
 1.1000e+00, 3.0700e+01, 7.5200e+01, 1.2000e+00, 1.0000e-01, 1.4600e+01,
 3.2400e+01, 8.5000e+00, 3.9100e+01, 4.0000e-01, nan, 4.0000e+00,
 nan, 1.7200e+01, 2.4100e+01, 1.9400e+01, nan, 2.0000e-01,
 1.4000e+00, 3.3000e+00, 1.1700e+01, 4.8000e+00, 5.9000e+00, 5.7000e+00,
 1.5200e+01, 3.9000e+00, 1.9900e+01, 2.3600e+01, nan, 8.8800e+01,
 1.0800e+01, 7.2000e+00, 4.3900e+01, 2.7000e+00, 2.3300e+01, 3.2600e+01,
 nan, 1.3300e+01, 2.5000e+00, 5.7600e+01, 2.6300e+01, 5.2000e+00,
 1.8000e+01, 5.0000e+00, 4.7000e+00, 1.0000e-01, 2.9000e+01, 6.4000e+00,
 1.4200e+01, 7.5000e+00, 3.8000e+00, 4.3700e+01, 2.0100e+01, 7.1000e+00,
 6.2900e+01, 2.7000e+00, 4.0300e+01, 4.0100e+01, 5.4000e+00, 3.0000e-01,

```

        nan, 6.4000e+00, 2.4000e+01, 1.2600e+01, 2.5000e+01, 1.3100e+01,
5.7000e+00, 1.8800e+01, 2.0000e-01, 1.3400e+01, 4.3000e+00, 4.2000e+01,
1.8800e+01, 2.5900e+01, 6.0000e+00, 2.6800e+01, 4.2000e+01, 3.9000e+00,
        nan, 3.2000e+00, 7.0000e+00, 2.8100e+01, 1.4300e+01, 1.2000e+01,
1.0300e+01,        nan, 2.3500e+01, 1.9100e+01, 1.1300e+01, 1.9200e+01,
2.0500e+01, 2.2000e+00, 2.2300e+01, 3.1600e+01,        nan, 9.8000e+00,
7.2000e+00, 1.6200e+01, 1.5300e+01,        nan, 5.0600e+01, 1.0000e-01,
3.9000e+00, 7.4000e+00, 1.2200e+01, 3.0000e-01, 3.4000e+00, 5.3000e+00,
2.3200e+01, 1.6000e+01,        nan, 1.1800e+01, 2.0000e-01, 1.4900e+01,
1.3820e+02, 9.4000e+00, 5.8000e+00, 1.5300e+01, 3.7000e+00, 1.3800e+01,
6.0000e+00,        nan, 1.5410e+02, 8.8000e+00,        nan, 3.4000e+00,
1.0000e+01, 1.6700e+01, 8.9000e+00, 3.0000e-01, 6.1000e+00, 1.3600e+01,
4.1000e+00, 7.6000e+00, 7.4000e+00, 1.5000e+01, 1.3420e+02, 7.3000e+00,
1.0500e+01,        nan, 7.2900e+01, 0.0000e+00, 7.7000e+00, 1.8000e+01,
1.3000e+01, 2.5000e+00, 3.2000e+00, 9.0700e+01, 9.3000e+00, 9.0000e+00,
2.9900e+01, 1.0000e-01, 1.1200e+01, 8.0000e+00, 1.6500e+01, 1.7400e+01,
1.2000e+01, 8.8000e+00, 3.8000e+00, 3.6000e+00, 1.6400e+01, 3.6000e+00,
2.5400e+01, 3.6000e+00, 1.3300e+01, 7.9000e+00, 5.6100e+01, 2.8500e+01,
1.3400e+01, 7.9000e+00,        nan, 1.0000e+00,        nan, 8.2000e+00,
4.0100e+01, 2.0000e-01,        nan, 8.5000e+00, 0.0000e+00, 3.5900e+01,
1.8300e+01, 9.4300e+01, 8.0000e-01, 1.2700e+01, 1.7000e+01, 6.9000e+00,
1.7500e+01, 6.3500e+01, 1.6600e+01, 6.2000e+00, 2.3000e+01, 2.7000e+00,
2.1300e+01, 1.8000e+00, 2.2900e+01, 1.6100e+01,        nan, 2.0000e+01,
1.1300e+01, 9.7800e+01, 7.8200e+01, 6.1000e+00, 5.0700e+01, 1.3000e+00,
8.0000e+00, 4.6000e+01, 3.7000e+00, 3.1000e+00, 5.0000e-01, 1.1700e+01,
1.3600e+01, 1.0800e+01, 5.0800e+01, 1.6000e+00, 4.3000e+00, 6.5000e+00,
4.4000e+00, 1.0000e-01, 3.4900e+01, 3.1400e+01, 2.8300e+01, 1.0400e+01,
8.8000e+00, 7.0600e+01, 1.8220e+02, 2.7800e+01, 2.9000e+00, 1.9500e+01,
7.4000e+00, 1.0000e+00, 1.5700e+01, 1.3100e+01], dtype=torch.float64),
'OS_STATUS': tensor([0., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
1., 0., 0., 0.,
        0., 0., 0., 0., 1., 0., 0., 1., 1., 0., 1., 0., 1., 1., 0., 0., 1., 1.,
1., 0., 1., 1., 1., 0., 1., 1., 0., 1., 0., 1., nan, 0., 0., 0., 0.,
0.,
        0., 0., 0., 0., 1., 1., 0., 1., 0., 0., 0., 0., 1., 1., 0., 0., 1., 0.,
0., 0., 0., 0., 0., 0., 0., 1., 1., 0., 0., 0., 0., 0., nan, 1., 1.,
0.,
        0., 0., 1., 1., 0., 1., 1., 0., 0., 1., 0., 0., 1., nan, 1., 1., 0.,
1.,
        1., 0., 1., 1., 1., 0., 0., 0., 1., 1., 1., 0., 0., 0., nan, 1., 1.,
0.,
        0., 0., 0., 1., 0., 1., 1., 0., 1., 1., 0., 0., 1., 0., 0., nan, 1.,
0.,
        0., 1., 1., 1., nan, 1., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
1.,
        1., 0., 1., 0., nan, 0., 1., 0., 0., 0., 1., 1., 1., 0., 0., 0., 0.,
0.,

```

```

0., 0., 0., 0., 0., 1., 1., 0., 0., 1., 1., 1., 1., 0., 0., 0., 1., 0.,
1., 0., 1., 0., 0., 1., 0., 0., 0., nan, 0., 0., 0., 1., 0., 1., 1.,
1.,
0., 0., 0., 0., 0., 1., 1., nan, 1., 0., 0., 0., 0., 1., 0., 0., 0.,
0.,
1., 0., 0., 1., 0., 0., 0., 1., 1., 0., 0., 1., 0., 0., 0., 0., 1., 0.,
1., 0., 0., 0., 0., 1., 0., 0., 0., 0., 1., 1., 1., nan, 1., 1., 0.,
0.,
nan, 1., 1., 0., 1., 0., 1., 0., 0., 1., 0., 0., 0., 0., nan, 1., 0.,
0.,
1., 1., nan, 0., 0., 0., 1., 1., 1., 0., 0., 0., 1., 1., 1., 0., nan,
nan,
0., 0., 1., 0., 1., nan, 1., 0., 0., 1., 0., 0., nan, 1., 1., 1., 0.,
0.,
1., 1., 0., 1., 0., 1., 1., 0., 0., 0., nan, 1., nan, 0., 1., 0., nan,
0.,
1., 1., 0., 0., 0., 0., 1., 0., 1., 0., nan, 0., 1., 0., 0., 0., 1.,
0.,
nan, 1., 0., 0., 0., 0., 1., 1., 0., 0., 1., 0., 0., 0., 0., 0., 0.,
0.,
1., 1., 0., 0., 0., 0., nan, 1., 0., 1., 0., 0., 0., 0., 0., 0., nan,
0.,
0., 1., 0., 0., 0., 0., nan, 1., 0., 0., 0., 0., 0., nan, 0., 0., 0.,
0.,
1., 1., 0., 1., nan, 0., 0., 0., 0., nan, 0., 0., 1., 0., 0., 0., 0.,
1.,
0., 0., nan, 0., 0., 0., 0., 0., 1., 1., 1., 1., 0., nan, 0., 1., nan,
0.,
0., 1., 1., 0., 1., 0., 0., 1., 1., 1., 1., 0., 0., nan, 0., 0., 1.,
0.,
0., 0., 1., 0., 0., 0., 0., 0., 1., 1., 0., 0., 1., 0., 0., 0., 0., 1.,
1., 0., 1., 0., 0., 0., 0., 1., nan, 1., nan, 0., 0., 0., nan, 0., 0.,
0.,
0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 1., 0., 0., 1., nan,
0.,
0., 0., 1., 0., 0., 0., 0., 0., 0., 1., 0., 1., 1., 1., 1., 0., 0., 1.,
1., 0., 1., 0., 0., 1., 1., 0., 0., 0., 1., 1., 0., 1., 0., 1.],
dtype=torch.float64),
'KARNOFSKY_PERFORMANCE_SCORE': tensor([ 90.,  nan, 100.,  nan, 100.,  nan,
nan, 100.,  60., 100., 100.,  90.,
      80.,  90.,  60.,  90.,  90.,  nan, 100., 100.,  90.,  nan,  80.,  90.,
      90.,  nan,  80.,  nan,  80.,  80.,  nan,  nan,  nan, 100., 100.,  nan,
      nan,  nan,  60.,  40.,  60.,  nan, 100.,  80.,  nan,  70.,  90.,  80.,
      nan,  70.,  80.,  nan,  nan,  90.,  90.,  80., 100.,  nan,  60.,  nan,
      nan,  80.,  90., 100., 100.,  80.,  nan,  80.,  nan,  nan,  80.,  90.,
     100.,  nan, 100.,  nan,  60.,  80.,  60.,  nan,  60.,  nan,  80.,  80.,
      90.,  nan,  nan,  nan,  80.,  90.,  60.,  nan,  60.,  80.,  nan,  nan,

```



```

nan, 60., nan, 80., 100., nan, 70., nan, 80., 60., 100., 60.,
80., 90., nan, nan, 60., 90., 90., 100., 80., 70., 90., nan,
80., nan, nan, nan, 60., 100., 80., 90., 80., 90., 100., 80.,
80., 60., nan, 100., nan, 90., nan, 60., nan, nan, 90., nan,
nan, nan, 90., nan, nan, 60., nan, 100., nan, nan, 90., nan,
nan, 100., 80., 80., nan, 90., 80., nan, 80., nan, nan, 100.,
nan, nan, 90., 80., 60., 100., 80., 20., nan, 100., nan, 100.,
nan, nan, nan, 80., 100., 100., nan, 60., nan, 60., nan, 90.,
60., 40., 60., nan, nan, 100., nan, 70., 90., nan, nan, nan,
80., nan, 80., nan, 80., 70., 90., 80., nan, 60., 80., 20.,
90., 100., 80., 60., 80., 80., nan, nan, nan, nan, 90., 90.,
80., nan, nan, nan, nan, 100., 60., 80., 80., nan, 90., nan,
nan, 90., 80., 90., nan, 90., 80., 80., nan, nan, 100., 60.,
80., nan, 80., nan, nan, 100., 80., 80., 80., 90., 80., 60.,
80., nan, nan, 90., nan, nan, nan, nan, nan, 40., 80., 80., nan,
80., nan, 90., nan, nan, 80., nan, 90., nan, 80., 100., 70.,
100., 90., nan, nan, nan, nan, 80., 80., 90., nan, nan, 70.,
80., 100., nan, 70., nan, nan, 100., nan, nan, 90., 80., nan,
80., 80., nan, 40., nan, nan, nan, 90., nan, 80., 80., 60.,
40., 90., 80., nan, nan, 80., 90., 90., nan, 80., nan, nan,
nan, 90., 80., 90., nan, nan, 80., 80., nan, 100., 100., 100.,
80., nan, 90., nan, nan, nan, 80., 60., 100., nan, 80., nan,
nan, 80., nan, 90., 100., 90., 80., 80., nan, nan, nan, nan,
70., 80., nan, 90., 90., nan, nan, 80., 80., 100., 80., nan,
nan, 80., 100., nan, 90., nan, 100., 90., 90., nan, nan, 100.,
90., 100., 90., 90., 90., nan, nan, 70., nan, 100., 100., 90.,
nan, nan, 90., 90., 80., nan, nan, 60., 100., 90., nan, 80.,
90., 80., 80., nan, nan, nan, nan, nan, nan, nan, 90., 40.,
nan, 70., nan, 100., nan, 70., nan, nan, 60., 90., nan, 60.,
nan, nan, nan, 80., nan, 100., 90., 80., nan, nan, nan, 80.,
70., 80., 100., nan, 100., 80., 90., nan, nan, 90., 90., nan,
100., 90., 80., 90., 90., 90., nan, nan, 40., 80., 100., 70.,
60., 90., 80., 100., 100., 60., 60., 50., nan, 80., nan, nan,
100., 80., nan, nan, nan, 60., nan, nan, nan, 70., nan, 90.,
100., 100., nan, nan, nan, 90., 90., nan, 100., 90., 80., nan,
50., nan, 80., 80., nan, 100., 100., nan, 100., 100., 100., 70.,
50., 90., nan, 80., nan, 80., nan, nan, nan, nan, nan, nan,
nan, nan, 80., nan, 90., 90., nan, 100., 70., 90., 40., nan,
nan, nan, 80., 100.], dtype=torch.float64),
'STUDY': tensor([0., 1., 0., 0., 1., 0., 0., 0., 0., 0., 0., 1., 0., 1.,
0., 0., 0.,
0., 0., 0., 0., 1., 0., 0., 1., 0., 0., 1., 1., 1., 0., 0., 0., 0., 1.,
1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 0., 1., 0., 0., 0., 0., 0., 0.,
0., 1., 0., 0., 1., 1., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 1., 1.,
0., 0., 0., 1., 1., 0., 1., 1., 1., 0., 1., 1., 0., 0., 1., 1., 0., 0.,
0., 0., 1., 1., 0., 1., 1., 1., 0., 1., 1., 0., 1., 0., 1., 0., 0., 0.,
1., 0., 1., 0., 1., 0., 0., 1., 1., 0., 0., 0., 0., 0., 0., 1., 1., 0.,

```

```

0., 0., 0., 0., 0., 1., 1., 1., 0., 1., 0., 0., 0., 1., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 1., 0., 1., 0., 0., 0., 0., 0., 0., 1., 1., 0., 0.,
1., 0., 1., 0., 1., 0., 0., 0., 0., 1., 1., 1., 1., 1., 0., 1., 1., 0.,
0., 0., 0., 1., 0., 1., 0., 1., 0., 1., 1., 0., 1., 1., 0., 1., 1., 0.,
1., 0., 0., 0., 1., 1., 0., 0., 0., 0., 1., 0., 0., 1., 0., 1., 1., 1.,
0., 0., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 1., 1., 0., 0., 1., 0.,
1., 1., 0., 0., 0., 1., 0., 0., 1., 1., 0., 0., 0., 1., 0., 0., 1., 1.,
1., 1., 0., 0., 0., 0., 1., 1., 1., 0., 1., 1., 1., 0., 0., 1., 0., 0.,
0., 1., 1., 1., 1., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0.,
1., 1., 0., 1., 0., 0., 0., 1., 0., 0., 0., 0., 1., 1., 1., 0., 0., 0.,
1., 0., 0., 0., 1., 0., 1., 0., 0., 1., 0., 0., 0., 0., 1., 1., 1., 1.,
1., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 1., 0., 0., 1., 0., 0., 0.,
1., 1., 1., 1., 1., 0., 1., 0., 0., 0., 0., 0., 1., 1., 0., 0., 0., 0.,
0., 1., 0., 0., 0., 0., 1., 1., 0., 0., 1., 0., 0., 1., 0., 0., 0., 0.,
0., 1., 1., 0., 1., 0., 0., 0., 0., 1., 0., 1., 0., 0., 0., 0., 1., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0.,
1., 1., 1., 0., 0., 1., 0., 1., 1., 0., 0., 0., 1., 1., 0., 0., 0., 0.,
0., 0., 0., 1., 0., 0., 0., 0., 0., 1., 1., 1., 1., 0., 0., 0., 0., 0.,
0., 1., 1., 0., 1., 1., 0., 1., 1., 0., 0., 1., 1., 0., 0., 0., 0., 0.,
0., 0., 1., 0., 0., 0., 0., 0., 1., 1., 0., 0., 1., 0., 0., 0., 0., 1.,
1., 0., 1., 1., 0., 0., 0., 1., 0., 1., 0., 0., 0., 0., 0., 1., 0., 0.,
1., 0., 0., 0., 0., 0., 0., 0., 1., 0., 1., 0., 0., 0., 0., 1., 0., 0.,
0., 0., 0., 1., 0., 0., 0., 0., 0., 1., 0., 1., 1., 1., 0., 0., 0., 0.,
1., 0., 1., 0., 0., 0., 1., 0., 0., 0., 1., 1., 0., 1., 0., 1.],
dtype=torch.float64),
'BCR_STATUS': tensor([1., 0., 0., 0., 0., 0., 1., 1., 0., 0., 1., 1., 0., 1.,
0., 0., 1., 0.,
1., 0., 1., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 1.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 1., 0., 0., 0., 0., 0.,
0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 1., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 1.,
1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0.,
0., 1., 1., 0., 0., 0., 0., 0., 0., 0., 1., 1., 0., 0., 0., 1., 0., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 1., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 1., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0.,
0., 1., 0., 0., 0., 1., 0., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 1., 0., 0., 0., 0., 0., 0., 1., 0., 0., 1., 0., 0., 0., 0., 0., 1.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,

```

```

0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 1., 1., 1., 0., 0., 0.,
0., 0., 0., 1., 0., 0., 1., 0., 1., 1., 0., 1., 0., 0., 0., 0., 0., 0.,
1., 0., 0., 1., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 1., 0., 0.,
0., 0., 1., 0., 1., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 1., 0.,
0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0.,
1., 1., 0., 0., 1., 1., 0., 0., 0., 0., 1., 0., 0., 0., 0., 1., 1., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 1., 1., 0., 0., 1., 0., 0., 1.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 1.,
1., 0., 0., 0., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 0., 0., 0., 1., 1., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
dtype=torch.float64),
'HISTOLOGICAL_DIAGNOSIS': tensor([2., 1., 3., 0., 1., 3., 3., 2., 3., 3., 0.,
2., 1., 3., 1., 2., 3., 3.,
0., 3., 3., 2., 1., 0., 3., 1., 2., 0., 1., 1., 1., 3., 3., 2., 3., 1.,
1., 2., 1., 1., 1., 3., 1., 1., 1., 3., 2., 1., nan, 0., 3., 3., 3.,
3.,
3., 1., 0., 2., 1., 1., 0., 0., 3., 3., 2., 0., 3., 1., 2., 3., 1., 1.,
0., 0., 3., 1., 1., 0., 1., 1., 1., 0., 1., 1., 2., 3., nan, 1., 0.,
2.,
0., 2., 1., 1., 3., 1., 1., 1., 2., 1., 1., 3., 1., nan, 1., 3., 3.,
2.,
1., 2., 1., 3., 1., 0., 2., 1., 1., 2., 2., 0., 3., 0., nan, 1., 1.,
0.,
2., 3., 0., 3., 2., 1., 1., 1., 0., 1., 0., 3., 2., 1., 3., nan, 2.,
0.,
0., 0., 2., 3., nan, 1., 3., 1., 3., 0., 0., 2., 0., 3., 1., 1., 0.,
0.,
1., 2., 1., 2., nan, 2., 3., 2., 3., 1., 1., 1., 1., 1., 3., 1., 1.,
2.,
2., 3., 3., 1., 3., 1., 0., 1., 3., 1., 1., 0., 1., 1., 0., 1., 1., 0.,
1., 0., 0., 2., 1., 1., 0., 0., 0., nan, 1., 3., 2., 1., 3., 1., 1.,
1.,
2., 0., 1., 1., 1., 1., 1., nan, 3., 0., 2., 0., 1., 1., 2., 3., 1.,
2.,
1., 1., 2., 3., 0., 1., 2., 2., 1., 1., 3., 0., 3., 1., 3., 2., 1., 1.,
1., 1., 0., 3., 3., 0., 1., 1., 1., 3., 1., 1., 1., nan, 3., 1., 2.,
2.,
nan, 1., 1., 1., 1., 3., 1., 3., 2., 3., 3., 3., 3., 0., nan, 1., 0.,
0.,
1., 1., nan, 1., 3., 0., 0., 1., 3., 0., 0., 3., 1., 1., 1., 3., nan,
nan,
1., 3., 3., 0., 1., nan, 1., 0., 0., 1., 3., 3., nan, 3., 1., 1., 1.,
1.,
1., 3., 2., 3., 0., 2., 2., 0., 3., 1., nan, 1., nan, 3., 1., 0., nan,
2.,
1., 1., 1., 1., 1., 0., 1., 2., 2., 2., nan, 3., 1., 1., 0., 3., 2.,
3.,

```

```

nan, 1., 3., 2., 2., 2., 1., 1., 0., 0., 1., 0., 0., 1., 0., 3., 2.,
0.,
0., 1., 1., 0., 1., 0., nan, 0., 0., 1., 3., 1., 3., 2., 2., 3., 1.,
2.,
2., 3., 3., 3., 0., 3., nan, 0., 0., 3., 1., 0., 0., nan, 0., 3., 0.,
0.,
1., 1., 1., 0., nan, 1., 0., 1., 1., nan, 0., 3., 1., 1., 0., 3., 0.,
0.,
3., 2., nan, 1., 3., 0., 3., 3., 0., 1., 1., 1., 1., nan, 2., 0., nan,
3.,
3., 1., 1., 0., 1., 1., 3., 1., 1., 0., 3., 1., 1., nan, 3., 0., 0.,
0.,
3., 2., 1., 3., 0., 0., 3., 0., 1., 1., 3., 2., 1., 3., 3., 0., 0., 1.,
1., 0., 1., 1., 2., 3., 2., 1., nan, 1., nan, 3., 3., 3., nan, 1., 3.,
0.,
1., 3., 0., 0., 2., 3., 3., 2., 1., 0., 1., 2., 3., 0., 2., 1., nan,
0.,
0., 2., 3., 1., 0., 3., 3., 2., 2., 1., 0., 1., 1., 1., 0., 3., 3., 0.,
1., 0., 1., 2., 3., 0., 1., 2., 3., 2., 1., 1., 2., 1., 0., 1.],
dtype=torch.float64),
'SEX': tensor([0., 0., 1., 0., 0., 1., 1., 1., 0., 0., 0., 1., 0., 0., 0., 1.,
1., 1.,
1., 0., 0., 1., 1., 0., 0., 0., 1., 0., 1., 0., 1., 1., 1., 1., 0., 1.,
1., 1., 0., 1., 1., 1., 1., 0., 0., 1., 1., 1., nan, 0., 1., 1., 1.,
0.,
0., 1., 0., 1., 1., 1., 0., 0., 0., 1., 1., 0., 1., 1., 0., 0., 1., 0.,
1., 1., 1., 0., 1., 0., 1., 0., 0., 1., 1., 0., 1., 1., nan, 1., 0.,
1.,
1., 0., 0., 1., 0., 1., 0., 1., 0., 0., 0., 1., 0., nan, 0., 0., 1.,
1.,
1., 1., 1., 1., 0., 1., 0., 1., 1., 0., 1., 1., 1., 0., nan, 0., 0.,
0.,
1., 0., 1., 1., 0., 1., 0., 1., 1., 1., 1., 0., 1., 1., 1., nan, 0.,
1.,
0., 0., 0., 1., nan, 1., 1., 1., 1., 1., 0., 1., 0., 0., 1., 0., 1.,
0.,
1., 0., 1., 1., nan, 0., 1., 0., 0., 0., 0., 1., 1., 1., 1., 1., 1.,
0.,
0., 1., 0., 1., 1., 0., 1., 0., 0., 1., 0., 0., 0., 1., 1., 1., 0., 1.,
1., 1., 1., 1., 1., 0., 0., 1., 1., nan, 1., 0., 0., 1., 1., 1., 0.,
1.,
1., 1., 1., 1., 1., 0., 0., nan, 1., 0., 1., 0., 1., 1., 1., 1., 0.,
0.,
1., 0., 1., 0., 1., 1., 1., 1., 0., 1., 0., 0., 1., 0., 1., 0., 1., 1.,
1., 1., 1., 1., 1., 0., 0., 1., 1., 1., 1., 0., 1., nan, 1., 1., 1.,
1.,
nan, 1., 1., 1., 1., 1., 1., 0., 0., 1., 1., 1., 1., 1., nan, 0., 1.,

```

```

1.,
nan,
1.,
1.,
0.,
0.,
1.,
1.,
1.,
1.,
0.,
nan,
1.,
0.,
0.,
0.,
1.,
1.,
0.,
1.,
1.,
0.,
1.,
nan,
1.,
1.,
1.,
0.,
1.,
1.,
nan,
1.,
0.,
1.,
1.,
1.,
1.,
1.,
1.,
1.,
1.,
0.,
1.,
1.,
0.,
0.,
0.,
1.,
1.,
1.,
1.,
1.,
nan,
0.,
0.,
1.,
1.,
1.,
0.,
1.,
0.,
1.,
1.,
0.,
1.,
1.,
1.,
nan,
0.,
0.,
1.,
1.,
1.,
nan,
1.,
0.,
0.,
1.,
1.,
1.,
0.,
nan,
1.,
0.,
1.,
1.,
nan,
1.,
0.,
1.,
1.,
0.,
0.,
1.,
1.,
0.,
1.,
0.,
1.,
0.,
1.,
1.,
0.,
1.,
0.,
1.,
1.,
0.,
1.,
nan,
1.,
1.,
1.,
1.,
0.,
0.,
0.,
1.,
0.,
1.,
0.,
1.,
1.,
1.,
1.,
0.,
0.,
1.,
0.,
0.,
0.,
1.,
1.,
1.,
1.,
1.,
0.,
0.,
1.,
0.,
0.,
1.,
1.,
0.,
0.,
1.,
1.,
nan,
1.,
nan,
1.,
1.,
1.,
nan,
0.,
1.,
1.,
0.,
0.,
1.,
0.,
1.,
0.,
0.,
1.,
1.,
1.,
1.,
1.,
0.,
0.,
1.,
nan,
0.,
0.,
0.,
0.,
1.,
1.,
1.,
1.,
1.,
0.,
0.,
0.,
0.,
1.,
1.,
0.,
0.,
0.,
0.,
1.,
1.,
0.,
1.,
1.,
1.,
0.,
1.,
1.,
1.,
0.,
0.,
0.,
1.,
0.,
0.,
1.,
0.,
1.],
dtype=torch.float64)}

```

```
[8]: test_dataset.ann
```

```

[8]: {'AGE': tensor([66., 39., 24., 43., 38., 19., 43., 33., 58., 39., 66., 40., 83.,
nan,
34., 70., 59., 47., 56., 33., nan, 62., 66., nan, 31., 53., 59., 30.,
56., nan, 59., nan, 62., 38., nan, 78., nan, nan, 60., 57., nan, 63.,
52., 44., 36., 61., 40., nan, 59., 73., nan, 30., 52., 55., 51., 24.,
59., 56., 36., 48., 41., nan, 74., 61., 59., 62., 21., 72., 63., 41.,
79., 29., 47., 43., 54., 32., 23., 52., nan, 62., 68., 33., 62., 59.,
35., 54., 37., nan, 57., 41., 59., 43., 52., 52., nan, 32., 73., 31.,
31., 38., 35., nan, 39., 89., 36., 38., 54., nan, 54., 65., 65., 41.,
62., 54., 74., 30., 63., nan, 48., 69., 41., 75., 42., nan, 54., 27.,
61., 36., 31., 34., 22., 61., 86., 78., 55., 37., nan, 23., 53., nan,
36., 57., 30., 57., nan, 44., 64., 30., 34., 63., 45., 60., 39., 73.,
33., nan, 30., 58., 64., 73., nan, 48., 28., 63., 49., 49., 48., 58.,
52., 83., 31., 28., nan, 35., 50., 61., 60., 24., 76., 29., 66., 35.,
20., 44., 65., 38., 50., 56., 39., 63., 57., 44., 58., 63., 53., 87.,

```

```

73., 47., 78., 76., 55., 58., 66., 52., 33., 24., 33., 35., 20., nan,
58., 32., 50., 71., 79., 72., nan, nan, 37., 74., 25., 62., 44., 66.,
34., 50., 31., 35., 38., 41., 34., 50., 29., 53., nan, 53., 55., 59.],
dtype=torch.float64),
'OS_MONTHS': tensor([4.8000e+00, 2.1600e+01, 2.0000e-01, 2.1000e+01,
1.8900e+01, 1.2000e+01,
4.9400e+01, 3.0000e+01, 9.3000e+00, 1.9200e+01, 2.5000e+00, 2.1500e+01,
1.8000e+00, nan, 7.9900e+01, 7.6000e+00, 3.1700e+01, 6.4600e+01,
8.0000e+00, 1.0000e-01, nan, 4.2700e+01, 1.5000e+01, nan,
7.5100e+01, 1.4200e+01, 1.1500e+01, 1.6000e+01, 4.7000e+00, nan,
6.8000e+00, nan, 1.5800e+01, 3.2000e+00, nan, 1.9000e+00,
nan, nan, 3.4900e+01, 5.8700e+01, nan, 1.3900e+01,
1.0900e+01, 2.5000e+00, 1.4000e+01, 8.0000e-01, 1.7400e+01, nan,
1.5000e+01, 1.7500e+01, nan, 1.7300e+01, 1.1300e+01, 6.0000e-01,
5.8000e+00, 1.6100e+01, 9.9000e+00, 1.0900e+01, 7.6000e+00, 4.2000e+00,
5.0100e+01, nan, 5.2000e+00, 6.7000e+00, 1.1500e+01, 1.0300e+01,
5.0500e+01, 1.2000e+00, 1.0000e-01, 3.7100e+01, 2.9000e+00, 3.0300e+01,
7.0000e+00, 6.6000e+00, 4.1000e+00, 1.5100e+01, 2.5500e+01, 3.7000e+00,
nan, 5.0000e-01, 1.2300e+01, 7.9000e+00, 3.4300e+01, 1.5400e+01,
7.7900e+01, 3.4300e+01, 1.8100e+01, nan, 5.1000e+00, 1.8400e+01,
1.6800e+01, 3.6500e+01, 2.0000e-01, 8.1000e+00, nan, 3.1800e+01,
3.7000e+00, 4.0800e+01, 1.9800e+01, 6.9000e+01, 2.3200e+01, nan,
6.8000e+00, 9.0000e-01, 8.3600e+01, 1.0000e-01, 6.2000e+00, nan,
3.7000e+01, 7.5000e+00, 1.6000e+00, 1.6000e+00, 1.5100e+01, 1.3500e+01,
1.0000e-01, 5.9000e+00, 3.2000e+00, nan, 1.7900e+01, 5.4000e+00,
1.5610e+02, 1.3600e+01, 4.1100e+01, nan, 3.2000e+00, 0.0000e+00,
7.8200e+01, 4.6000e+00, 1.8900e+01, 7.2500e+01, 2.0000e-01, 5.7000e+00,
6.9000e+00, 2.7000e+00, 9.7000e+00, 1.4200e+01, nan, 2.0000e-01,
2.3000e+00, nan, 6.4000e+00, 3.4000e+00, 9.4000e+00, 1.9600e+01,
nan, 7.7000e+00, 1.2050e+02, 6.6000e+00, 1.4500e+01, 4.6000e+00,
4.7000e+00, 5.7000e+00, 7.6000e+00, 1.9000e+00, 1.4000e+01, nan,
3.3600e+01, 2.0800e+01, 1.2700e+01, 2.1000e+00, nan, 1.4600e+01,
9.9000e+00, 1.7100e+01, 7.8000e+00, 2.5300e+01, 2.3800e+01, 6.0000e-01,
1.5600e+01, 4.9000e+00, 4.7000e+00, 1.6500e+01, nan, 9.4500e+01,
1.0790e+02, 1.0400e+01, 5.0000e+00, 9.4600e+01, 4.7000e+00, 1.0000e-01,
5.4000e+00, 1.2440e+02, 1.0000e-01, 2.7000e+01, 2.3000e+00, 8.3000e+00,
1.0200e+01, 3.0000e-01, 1.2600e+01, 1.7500e+01, 1.2900e+01, 1.5000e+00,
3.9000e+00, 2.6700e+01, 1.2600e+01, 1.1400e+01, 7.8000e+00, 1.6100e+01,
1.4700e+01, 7.4000e+00, 1.5400e+01, 4.7000e+00, 8.0000e+00, 1.7500e+01,
4.1000e+00, 1.3370e+02, 3.5000e+00, 8.0000e-01, 1.5400e+01, nan,
7.9000e+00, 2.7000e+01, 2.2500e+01, 1.1300e+01, 3.1000e+00, 8.9000e+00,
nan, nan, 7.4000e+00, 7.2000e+00, 7.7000e+00, 2.4000e+00,
2.2600e+01, 2.5000e+00, 1.0400e+01, 2.7000e+00, 1.9600e+01, 1.8200e+01,
1.3070e+02, 4.8400e+01, 3.3000e+01, 1.0600e+01, 3.0700e+01, 1.0800e+01,
nan, 2.2400e+01, 1.6200e+01, 5.4200e+01], dtype=torch.float64),
'OS_STATUS': tensor([1., 0., 0., 1., 1., 0., 0., 1., 0., 0., 1., 0., 1., nan,
1., 1., 0., 0.,

```

```

0., 0., nan, 0., 1., nan, 1., 0., 1., 0., 0., nan, 0., nan, 1., 0.,
nan, 1.,
nan, nan, 1., 0., nan, 1., 0., 1., 1., 1., 0., nan, 1., 0., nan, 0.,
0., 0.,
1., 0., 1., 0., 0., 0., 1., nan, 0., 1., 0., 1., 1., 1., 0., 0., 1.,
0.,
0., 0., 0., 0., 1., 0., nan, 0., 1., 1., 0., 1., 0., 0., 0., nan, 1.,
0.,
1., 0., 0., 0., nan, 0., 1., 1., 1., 0., 0., nan, 0., 1., 0., 0., 0.,
nan,
0., 0., 0., 0., 0., 0., 0., 0., 1., nan, 0., 1., 0., 1., 1., nan, 1.,
0.,
0., 0., 0., 0., 0., 0., 1., 1., 0., 0., nan, 0., 1., nan, 0., 0., 0.,
0.,
nan, 0., 1., 0., 1., 0., 0., 1., 0., 0., 0., nan, 1., 1., 1., 1., nan,
0.,
0., 1., 0., 1., 0., 0., 0., 1., 0., 0., nan, 1., 0., 1., 0., 0., 1.,
0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 1., 0., 1., 0., 1., 0., 0.,
1., 1., 1., 1., 1., 1., 0., 1., 0., 0., 0., nan, 1., 0., 0., 1., 1.,
0.,
nan, nan, 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 1., 0., 0., 0., 0.,
0.,
nan, 1., 0., 0.], dtype=torch.float64),
'KARNOFSKY_PERFORMANCE_SCORE': tensor([ 60., 100., nan, 90., 80., 90.,
90., 80., 80., nan, 80., nan,
nan, nan, 100., 80., nan, 100., 100., nan, nan, nan, 100., nan,
90., 70., 80., nan, 100., nan, 100., nan, 80., 90., nan, nan,
nan, nan, 80., 80., nan, nan, 100., nan, 80., 40., 90., nan,
nan, 90., nan, nan, 100., nan, nan, 90., 100., 90., 80., 90.,
100., nan, 60., 80., nan, 80., nan, nan, nan, nan, 40., 90.,
90., nan, 90., 80., 90., nan, nan, nan, nan, 60., nan, nan,
90., 100., 50., nan, 80., nan, nan, nan, nan, 100., nan, nan,
nan, 90., 100., 100., nan, nan, nan, 60., nan, 80., nan, nan,
100., 80., 60., 80., 100., 70., nan, nan, 40., nan, 90., 60.,
nan, nan, 90., nan, 80., nan, nan, 80., 80., 100., nan, 80.,
60., 60., 40., 90., nan, 100., nan, nan, 90., 100., 100., 100.,
nan, 80., nan, nan, 80., 40., 80., 80., 90., 40., 90., nan,
100., nan, nan, nan, nan, nan, nan, 100., 80., 100., nan, 90., nan,
70., 60., 90., nan, nan, 90., nan, 80., nan, 100., 80., nan,
nan, 100., nan, nan, 80., nan, nan, 80., 80., 80., 60., nan,
90., nan, nan, nan, nan, 60., 80., nan, nan, nan, nan, 60.,
80., 100., 100., 70., 100., nan, 100., nan, 100., 70., nan, 80.,
nan, nan, nan, 80., nan, nan, 100., nan, 80., nan, nan, 100.,
90., 100., nan, 90., 90., nan, nan, nan, nan, 90.],
dtype=torch.float64),
'STUDY': tensor([1., 0., 0., 0., 0., 0., 0., 1., 1., 0., 1., 0., 1., 0., 0.,

```

```

1., 0., 0.,
    0., 0., 0., 0., 1., 0., 0., 1., 1., 0., 0., 0., 1., 0., 1., 0., 1., 1.,
    0., 0., 1., 1., 0., 1., 0., 1., 1., 1., 0., 0., 1., 1., 0., 0., 1., 0.,
    1., 0., 1., 0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1., 0., 1., 0.,
    0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 1., 0., 0., 0., 1., 1., 0.,
    0., 0., 0., 0., 0., 0., 1., 0., 1., 0., 0., 0., 1., 1., 0., 0., 0., 0.,
    1., 1., 1., 0., 0., 0., 0., 0., 1., 0., 0., 1., 0., 1., 0., 0., 1., 0.,
    0., 0., 0., 0., 0., 0., 1., 1., 1., 0., 0., 0., 1., 0., 0., 0., 0., 0.,
    0., 1., 1., 0., 1., 1., 0., 1., 0., 0., 0., 0., 1., 1., 0., 1., 1., 0.,
    0., 1., 1., 1., 0., 0., 1., 1., 0., 0., 0., 0., 0., 1., 1., 0., 1., 0.,
    1., 0., 0., 0., 1., 0., 0., 0., 0., 0., 1., 1., 0., 1., 0., 0., 1., 0.,
    1., 1., 1., 1., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 1., 1.,
    1., 0., 0., 1., 0., 0., 0., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0.,
    0., 0., 0., 0.], dtype=torch.float64),
'BCR_STATUS': tensor([0., 1., 0., 0., 0., 1., 1., 0., 0., 0., 0., 0., 0., 1.,
0., 0., 0., 0.,
    0., 0., 1., 0., 0., 1., 0., 0., 0., 0., 1., 1., 0., 0., 0., 1., 0., 0.,
    0., 1., 0., 0., 1., 0., 1., 0., 0., 0., 0., 1., 0., 0., 1., 0., 0., 0.,
    0., 0., 0., 0., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
    0., 0., 1., 1., 0., 0., 1., 0., 0., 1., 0., 0., 0., 1., 0., 0., 0., 0.,
    0., 0., 1., 1., 1., 0., 0., 1., 0., 1., 1., 0., 0., 0., 1., 1., 0., 1.,
    0., 0., 0., 1., 0., 1., 0., 1., 0., 1., 0., 0., 0., 0., 1., 0., 1.,
    0., 1., 1., 1., 0., 0., 0., 0., 0., 1., 1., 0., 1., 0., 1., 1., 1.,
    0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 1., 0., 0., 0., 0., 0., 0.,
    1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0.,
    0., 0., 0., 1., 0., 0., 0., 0., 1., 1., 0., 0., 1., 0., 0., 0., 0., 0.,
    0., 0., 0., 0., 1., 0., 1., 0., 1., 1., 0., 0., 1., 0., 1., 0., 0., 0.,
    1., 1., 0., 0., 0., 0., 1., 0., 0., 0., 0., 1., 0., 1., 0., 0., 0., 0.,
    1., 0., 0., 0.], dtype=torch.float64),
'HISTOLOGICAL_DIAGNOSIS': tensor([1., 0., 0., 3., 0., 2., 0., 1., 1., 0., 1.,
2., 1., nan, 0., 1., 3., 3.,
    3., 3., nan, 2., 1., nan, 2., 1., 1., 0., 3., nan, 1., nan, 1., 2.,
nan, 1.,
    nan, nan, 1., 1., nan, 1., 0., 1., 1., 1., 2., nan, 1., 1., nan, 0.,
1., 0.,
    1., 3., 1., 0., 0., 0., 0., nan, 2., 2., 1., 1., 1., 1., 1., 3., 1.,
0.,
    2., 3., 3., 0., 0., 2., nan, 3., 1., 0., 3., 1., 2., 3., 0., nan, 1.,
0.,
    2., 2., 0., 3., nan, 3., 1., 2., 1., 0., 3., nan, 1., 1., 0., 2., 3.,
nan,
    1., 1., 1., 3., 3., 3., 3., 0., 1., nan, 0., 1., 3., 1., 0., nan, 1.,
3.,
    2., 3., 0., 3., 2., 0., 1., 1., 1., 0., nan, 2., 1., nan, 0., 2., 3.,
0.,
    nan, 1., 1., 0., 1., 1., 0., 1., 0., 3., 0., nan, 1., 1., 0., 1., nan,
3.,

```



```

0., 1., 1., 1., 0., 2., 1., 1., 0., 2., nan, 3., 0., 1., 1., 0., 1.,
2.,
1., 2., 3., 2., 1., 2., 3., 3., 2., 3., 1., 1., 2., 1., 2., 3., 1., 3.,
1., 1., 1., 1., 0., 1., 3., 3., 3., 3., 2., nan, 0., 0., 3., 1., 1.,
1.,
nan, nan, 0., 1., 2., 0., 0., 1., 1., 1., 0., 2., 2., 0., 3., 0., 0.,
2.,
nan, 3., 0., 3.], dtype=torch.float64),
'SEX': tensor([1., 0., 0., 1., 0., 1., 0., 0., 1., 1., 0., 0., 1., nan, 1., 1.,
1., 0.,
0., 0., nan, 1., 1., nan, 0., 1., 0., 0., 0., nan, 1., nan, 1., 1.,
nan, 0.,
nan, nan, 1., 0., nan, 1., 1., 1., 1., 1., 0., nan, 0., 0., nan, 0.,
0., 1.,
1., 0., 0., 0., 0., 0., 1., nan, 0., 0., 0., 1., 1., 1., 0., 0., 0.,
1.,
0., 1., 1., 1., 1., 1., nan, 1., 1., 0., 1., 1., 0., 0., 1., nan, 1.,
0.,
0., 1., 0., 0., nan, 1., 1., 1., 1., 1., 0., nan, 1., 1., 1., 0., 1.,
nan,
1., 1., 0., 0., 0., 1., 1., 1., 0., nan, 1., 0., 0., 0., 1., nan, 1.,
1.,
1., 1., 1., 1., 1., 0., 1., 1., 0., 1., nan, 0., 0., nan, 0., 0., 0.,
1.,
nan, 0., 0., 1., 0., 1., 0., 1., 0., 0., 0., nan, 1., 0., 1., 0., nan,
1.,
1., 1., 0., 1., 1., 1., 1., 1., 0., 1., nan, 0., 1., 1., 1., 1., 0.,
1.,
1., 1., 0., 1., 0., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 1., 0., 1.,
1.,
1., 0., 1., 1., 1., 0., 1., 0., 1., 0., 1., nan, 1., 1., 1., 1., 1.,
0.,
nan, nan, 0., 1., 1., 1., 1., 0., 1., 1., 1., 0., 0., 0., 0., 1., 1.,
0.,
nan, 1., 1., 0.], dtype=torch.float64)}

```

- Make a histogram plot of the follow up times in months (OS_MONTHS) (use sns.histplot)

```

[9]: os_months_train = train_dataset.ann['OS_MONTHS']
os_months_test = test_dataset.ann['OS_MONTHS']

plt.figure(figsize=(10, 6))

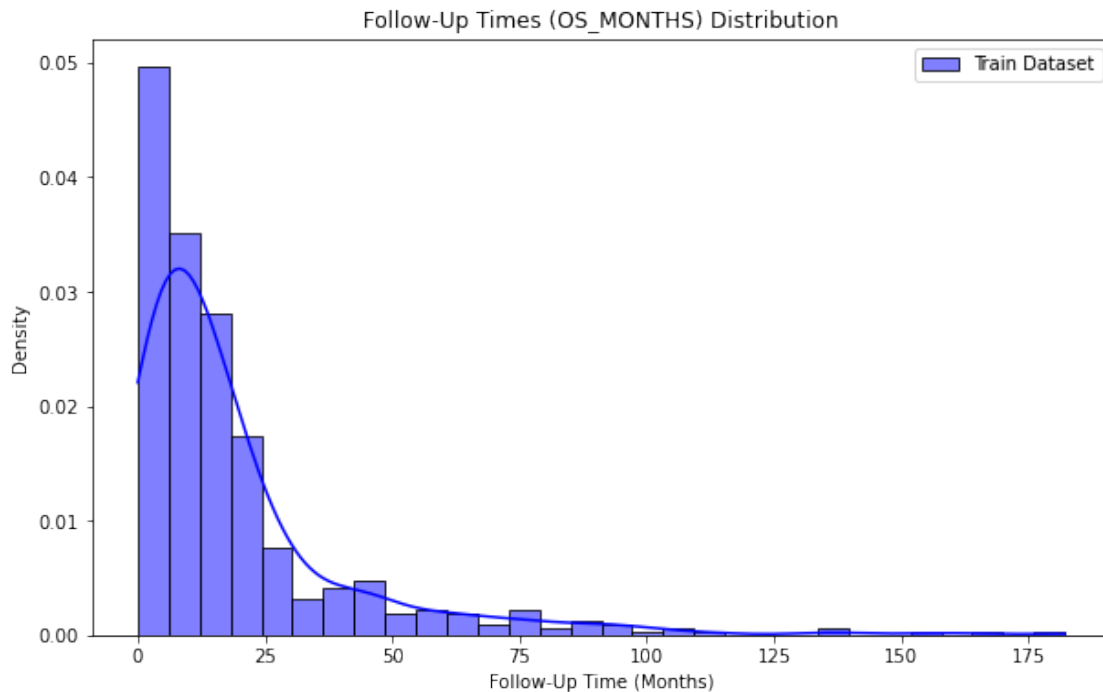
sns.histplot(os_months_train, kde=True, color='blue', label='Train Dataset',
             bins=30, stat='density', linewidth=0.8)

plt.title('Follow-Up Times (OS_MONTHS) Distribution')
plt.xlabel('Follow-Up Time (Months)')

```

```
plt.ylabel('Density')
plt.legend()
```

[9]: <matplotlib.legend.Legend at 0x7fca923dbcd0>



[]:

- Make a histogram of the age distribution of the patients in the training data; facet the histogram by “SEX” variable (see flexynesis.utils.plot_boxplot)

```
[54]: age_train = train_dataset.ann['AGE']
sex_train = train_dataset.ann['SEX']

data1 = pd.DataFrame({'Age': age_train, 'Sex': sex_train})

sns.boxplot(x='Sex', y='Age', data = data1, palette='Set2')
plt.title('Age Distribution by Sex')
plt.show()
```

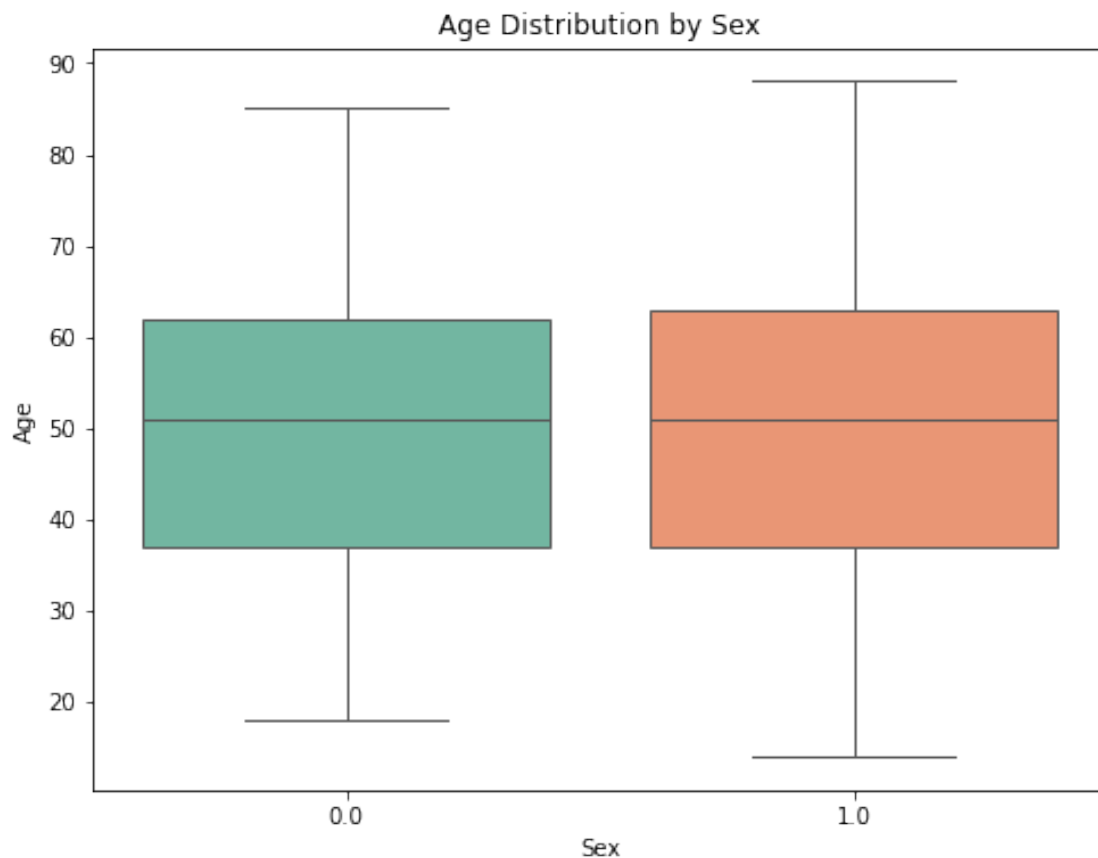
/tmp/ipykernel_165/2383391392.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

/gnu/store/88qdfdp9h0003w4wp1rv46khc3dpp8j6-profile/lib/python3.10/site-
packages/seaborn/_base.py:949: FutureWarning: When grouping with a length-1
list-like, you will need to pass a length-1 tuple to get_group in a future
version of pandas. Pass `(name,)` instead of `name` to silence this warning.
/gnu/store/88qdfdp9h0003w4wp1rv46khc3dpp8j6-profile/lib/python3.10/site-
packages/seaborn/categorical.py:640: FutureWarning: SeriesGroupBy.grouper is
deprecated and will be removed in a future version of pandas.
/gnu/store/88qdfdp9h0003w4wp1rv46khc3dpp8j6-profile/lib/python3.10/site-
packages/seaborn/_base.py:949: FutureWarning: When grouping with a length-1
list-like, you will need to pass a length-1 tuple to get_group in a future
version of pandas. Pass `(name,)` instead of `name` to silence this warning.
/gnu/store/88qdfdp9h0003w4wp1rv46khc3dpp8j6-profile/lib/python3.10/site-
packages/seaborn/categorical.py:640: FutureWarning: SeriesGroupBy.grouper is
deprecated and will be removed in a future version of pandas.

```



- Make a summary of all available clinical variables (see `flexynesis.print_summary_stats`)

```
[12]: flexynesis.print_summary_stats(train_dataset)
```

Summary for variable: AGE

Numerical Variable Summary: Median = 51.0, Mean = 50.33781190019194

```

-----
Summary for variable: OS_MONTHS
Numerical Variable Summary: Median = 11.6, Mean = 19.090978886756233
-----
Summary for variable: OS_STATUS
Numerical Variable Summary: Median = 0.0, Mean = 0.36153846153846153
-----
Summary for variable: KARNOFSKY_PERFORMANCE_SCORE
Numerical Variable Summary: Median = 80.0, Mean = 82.45454545454545
-----
Summary for variable: STUDY
Categorical Variable Summary:
  Label: Brain Lower Grade Glioma, Count: 353
  Label: Glioblastoma multiforme, Count: 203
-----
Summary for variable: BCR_STATUS
Categorical Variable Summary:
  Label: IGC, Count: 454
  Label: NCH, Count: 102
-----
Summary for variable: HISTOLOGICAL_DIAGNOSIS
Categorical Variable Summary:
  Label: astrocytoma, Count: 115
  Label: glioblastoma, Count: 201
  Label: oligoastrocytoma, Count: 79
  Label: oligodendroglioma, Count: 126
  Label: nan, Count: 35
-----
Summary for variable: SEX
Categorical Variable Summary:
  Label: Female, Count: 209
  Label: Male, Count: 312
  Label: nan, Count: 35
-----

```

- Notice that the categorical variables such as “SEX”, “STUDY”, “HISTOLOGICAL_DIAGNOSIS” are encoded numerically in the “dataset.ann” objects. Use dataset.label_mappings to map the STUDY variable to their original labels. Print the top 10 values in dataset.ann[‘STUDY’] and the mapped label values.

```

[13]: study_values = train_dataset.ann['STUDY'].numpy()

study_labels = train_dataset.label_mappings['STUDY']

top_10_study_values = study_values[:10]
top_10_study_labels = [study_labels[val] for val in top_10_study_values]

print("Top 10 numerical values in 'STUDY':", top_10_study_values)

```

```
print("Top 10 mapped tags for 'STUDY':", top_10_study_labels)
```

Top 10 numerical values in 'STUDY': [0. 1. 0. 0. 1. 0. 0. 0. 0. 0.]
 Top 10 mapped tags for 'STUDY': ['Brain Lower Grade Glioma', 'Glioblastoma multiforme', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma', 'Glioblastoma multiforme', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma']

- Now, let's explore the data matrices. Make a PCA plot of the mutation data matrix and color the samples by "HISTOLOGICAL_DIAGNOSIS". See `flexynesis.plot_dim_reduced` function

First create a pandas data frame with the data matrix of interest with feature and sample names > `df = pd.DataFrame(train_dataset.dat['cna'], index = train_dataset.samples, columns=train_dataset.features['cna'])`

Check the data frame contents > `df.head()`

Make a PCA plot of CNA values using the labels from the STUDY variable

Note: if you couldn't map the labels above, you can also use `train_dataset.dat['STUDY']` as labels

```
[14]: df_mutation = pd.DataFrame(train_dataset.dat['mut'], index=train_dataset.
    ↪samples, columns=train_dataset.features['mut'])

print(df_mutation.head())
```

	IDH2	IDH1	RELN	ATRX	PIK3CA	EGFR	\
TCGA-TM-A84H	-0.148522	0.982173	-0.172133	1.707482	-0.307389	-0.331331	
TCGA-06-0650	-0.148522	-1.018150	-0.172133	-0.585658	-0.307389	-0.331331	
TCGA-E1-A7Y0	-0.148522	0.982173	-0.172133	-0.585658	-0.307389	-0.331331	
TCGA-DU-7290	-0.148522	-1.018150	-0.172133	-0.585658	-0.307389	-0.331331	
TCGA-06-0168	-0.148522	-1.018150	-0.172133	-0.585658	-0.307389	-0.331331	

	TP53	COL6A3	TEKT4	SVIL	...	SIGLEC12	ZNF571	\
TCGA-TM-A84H	1.235829	-0.177595	-0.09526	-0.104447	...	-0.042448	-0.042448	
TCGA-06-0650	-0.809174	-0.177595	-0.09526	-0.104447	...	-0.042448	-0.042448	
TCGA-E1-A7Y0	-0.809174	-0.177595	-0.09526	-0.104447	...	-0.042448	-0.042448	
TCGA-DU-7290	-0.809174	-0.177595	-0.09526	-0.104447	...	-0.042448	-0.042448	
TCGA-06-0168	-0.809174	-0.177595	-0.09526	-0.104447	...	-0.042448	-0.042448	

	NEURL1	NYAP2	CAMKK1	SMARCA2	NAGA	PTPN6	\
TCGA-TM-A84H	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	
TCGA-06-0650	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	
TCGA-E1-A7Y0	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	
TCGA-DU-7290	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	
TCGA-06-0168	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	-0.042448	

	SEPT12	SPG7
TCGA-TM-A84H	-0.042448	-0.060084
TCGA-06-0650	-0.042448	-0.060084

```
TCGA-E1-A7Y0 -0.042448 -0.060084
TCGA-DU-7290 -0.042448 -0.060084
TCGA-06-0168 -0.042448 -0.060084
```

```
[5 rows x 321 columns]
```

```
[15]: imputer = SimpleImputer(strategy='mean')

df_mutation_imputed = pd.DataFrame(imputer.fit_transform(df_mutation),
    ↪ columns=df_mutation.columns)

print(df_mutation_imputed.isna().sum())
```

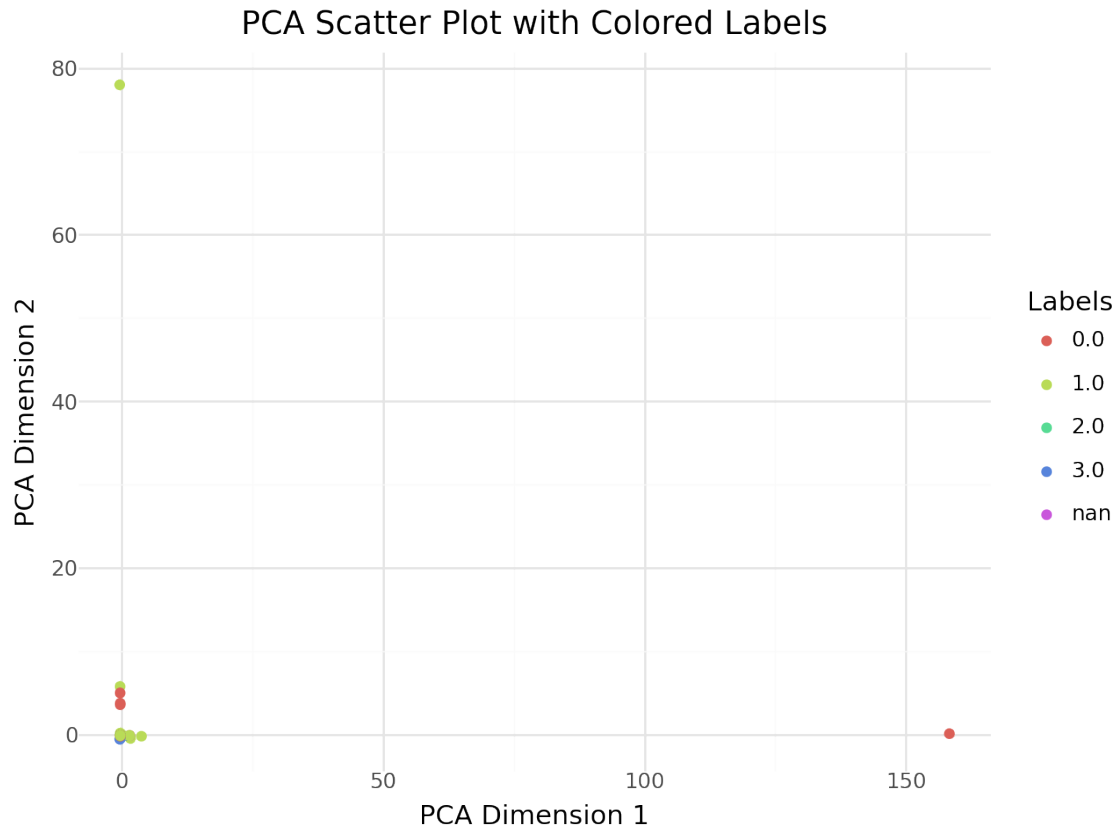
```
IDH2      0
IDH1      0
RELN      0
ATRX      0
PIK3CA    0
..
SMARCA2   0
NAGA      0
PTPN6     0
SEPT12    0
SPG7      0
Length: 321, dtype: int64
```

```
[16]: pca = PCA(n_components=2)
pca_result = pca.fit_transform(df_mutation_imputed)

pca_df = pd.DataFrame(pca_result, columns=["PCA1", "PCA2"], index=train_dataset.
    ↪ samples)
```

```
[19]: histological_diagnosis = train_dataset.ann['HISTOLOGICAL_DIAGNOSIS'].numpy()

flexynesis.plot_dim_reduced(pca_df, labels=histological_diagnosis,)
```



- (Optional exercise ideas):
 - Make a PCA plot coloring the samples by HISTOLOGICAL_DIAGNOSIS, GENDER, or any other clinical variable
 - Repeat the same exercise on the mutation data matrix.

1.6 2. Training a single model using manually set hyperparameters

Now that we have familiarized ourselves with the dataset at hand, we can start building models.

First we will do a single model training by manually setting hyperparameters. Based on the model performance, we will try modifying individual hyperparameters and build more and more models and see if we can improve model performance.

We will need to define the following components for starting a model training:

1. Split the `train_dataset` into train/validation components
2. Define data loaders for both train and validation splits
3. Define a `pytorch-lightning` trainer
4. Define a model with hyperparameters
5. Fit the model

```
[20]: # randomly assign 80% of samples for training, 20% for validation
```

```

train_indices = random.sample(range(0, len(train_dataset)),
    ↪int(len(train_dataset) * 0.8))
val_indices = list(set(range(len(train_dataset))) - set(train_indices))
train_subset = train_dataset.subset(train_indices)
val_subset = train_dataset.subset(val_indices)

# define data loaders for train/validation splits
from torch.utils.data import DataLoader
train_loader = DataLoader(train_subset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_subset, batch_size=32, shuffle = False)

```

Now, we need to define a model with manually set hyperparameters and a lightning-trainer fit the model.

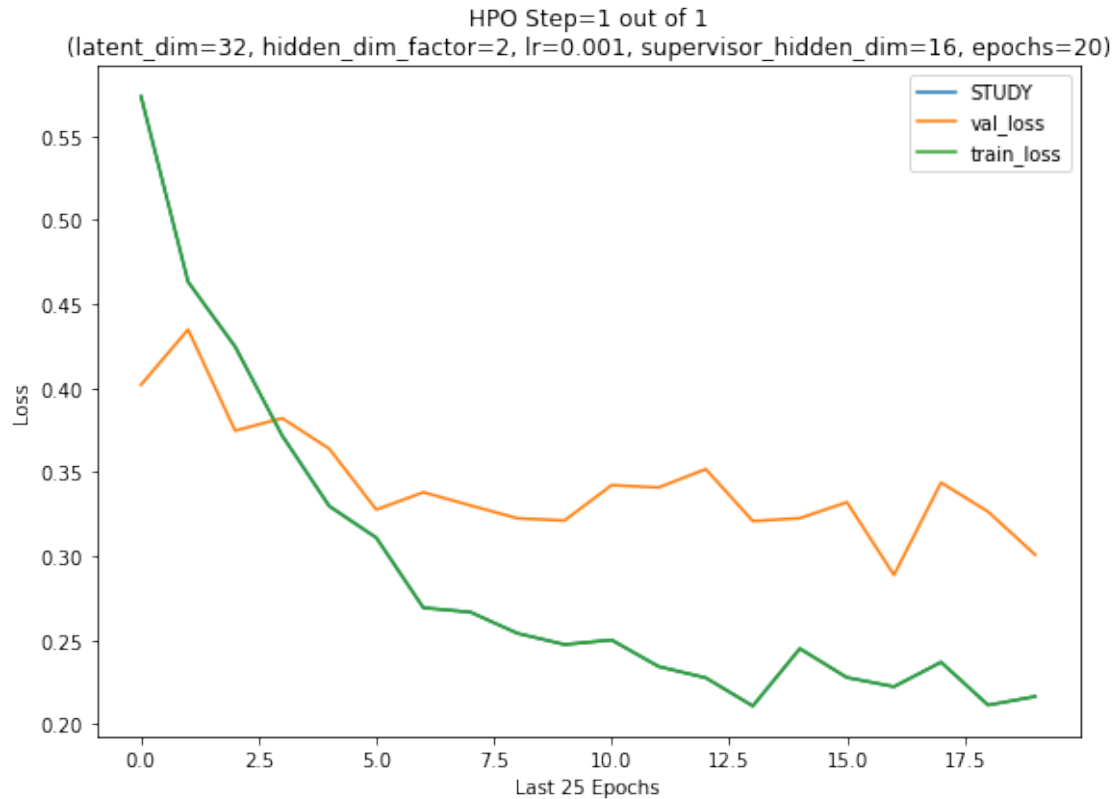
Notice: Notice the callback we are passing to the trainer which enables us to plot the loss values as the training progresses.

```

[21]: # Define a model with manually set hyperparameters for the DirectPred model
myparams = {'latent_dim': 32, 'hidden_dim_factor': 2, 'lr': 0.001,
    ↪'supervisor_hidden_dim': 16, 'epochs': 20}
model = flexynesis.DirectPred(config = myparams, dataset = train_dataset,
    ↪target_variables=['STUDY'])
trainer = pl.Trainer(max_epochs=myparams['epochs'], default_root_dir=".",
    ↪logger=False, enable_checkpointing=False,
    ↪callbacks=[flexynesis.LiveLossPlot(myparams, 1, 1)])

# Fit the model
trainer.fit(model, train_loader, val_loader)

```

`Trainer.fit` stopped: `max\_epochs=20` reached.

While we can observe how well the model training went based on the “loss” values, we can also evaluate the model performance on test dataset

```
[22]: # evaluate the model performance on predicting the target variable
flexynesis.evaluate_wrapper("DirectPred", model.predict(test_dataset),
                             ↪test_dataset)
```

```
[22]:
```

	method	var	variable_type	metric	value
0	DirectPred	STUDY	categorical	balanced_acc	0.765359
1	DirectPred	STUDY	categorical	f1_score	0.785429
2	DirectPred	STUDY	categorical	kappa	0.532110
3	DirectPred	STUDY	categorical	average_auroc	0.849443
4	DirectPred	STUDY	categorical	average_aupr	0.718645

1.7 2.1 Exercise

- Now, repeat the above model training and evaluation by manually changing the hyperparameters (Try at least 5 different combinations)
- See if you can find a better hyperparameter combination that yields a better classification performance than the initial setup we provided.

- See the default hyperparameter ranges we use for Flexynesis here: <https://github.com/BIMSBbioinfo/flexynesis/blob/69b92ca9370551e9fcc82a756cb42c72bef4a4b1/flexynesis/> but feel free to try outside these ranges too.
- Also try to observe the impact of the changing parameters on how the train/validation loss curves change.

```
myparams = {'latent_dim': XX, 'hidden_dim_factor': XX, 'lr': XX, 'supervisor_hidden_dim': XX,

model = flexynesis.DirectPred(config = myparams, dataset = train_dataset, target_variables=['S

trainer = pl.Trainer(max_epochs=myparams['epochs'], default_root_dir=".", logger=False, enable

callbacks=[flexynesis.LiveLossPlot(myparams, 1, 1)])

trainer.fit(model, train_loader, val_loader)

flexynesis.evaluate_wrapper("DirectPred", model.predict(test_dataset), test_dataset)
```

```
[23]: hyperparameter_sets = [
    {'latent_dim': 32, 'hidden_dim_factor': 2, 'lr': 0.001,
    ↪ 'supervisor_hidden_dim': 16, 'epochs': 20},
    {'latent_dim': 64, 'hidden_dim_factor': 4, 'lr': 0.01,
    ↪ 'supervisor_hidden_dim': 32, 'epochs': 50},
    {'latent_dim': 128, 'hidden_dim_factor': 8, 'lr': 0.0001,
    ↪ 'supervisor_hidden_dim': 64, 'epochs': 30},
    {'latent_dim': 16, 'hidden_dim_factor': 1, 'lr': 0.005,
    ↪ 'supervisor_hidden_dim': 8, 'epochs': 25},
    {'latent_dim': 64, 'hidden_dim_factor': 16, 'lr': 0.0005,
    ↪ 'supervisor_hidden_dim': 64, 'epochs': 40}
]
```

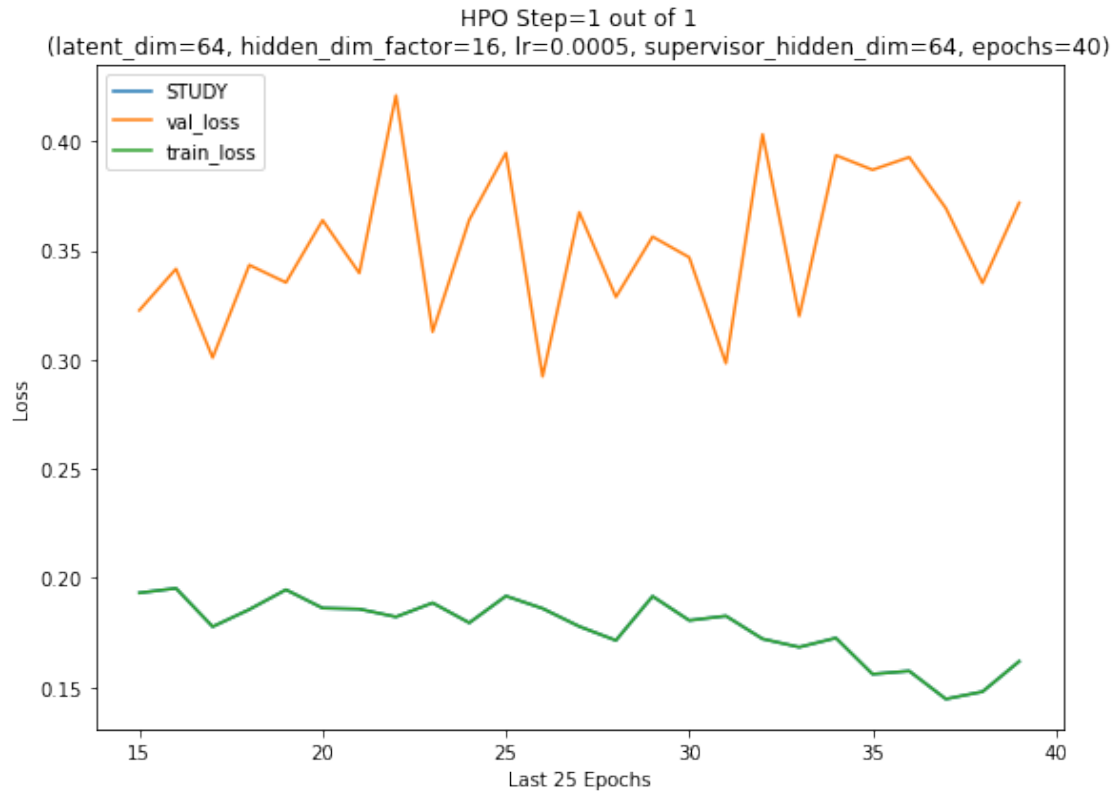
```
[24]: for params in hyperparameter_sets:
    print(f"Training with parameters: {params}")

    # Initialize the model with the current hyperparameters
    model = flexynesis.DirectPred(config=params, dataset=train_dataset,
    ↪ target_variables=['STUDY'])

    # Setup PyTorch Lightning Trainer
    trainer = pl.Trainer(
        max_epochs=params['epochs'], # Set number of epochs
        default_root_dir=".", # Specify directory if needed
        logger=False, # Disable logger
        enable_checkpointing=False, # Disable checkpointing
        callbacks=[flexynesis.LiveLossPlot(params, 1, 1)] # Visualize loss
    ↪ curves
    )
```

```
# Train the model
trainer.fit(model, train_loader, val_loader)

# Evaluate the model on the test dataset
test_predictions = model.predict(test_dataset)
flexynesis.evaluate_wrapper("DirectPred", test_predictions, test_dataset)
```



`Trainer.fit` stopped: `max\_epochs=40` reached.

Warning!!: In reality, we don't select the best models based on performance on the test dataset.

The best model is selected based on the validation loss value, where the model parameters that yields the lowest validation loss is selected to be the best model.

The validation dataset which we use to compute the validation loss is basically a subset of the training dataset.

1.8 3. Automating the Hyperparameter Optimisation Procedure

What we did in the above section was to set random hyperparameters, build a model, evaluate the model and try different hyperparameters based on our previous model performance. However,

this process can be quite time consuming and arbitrary. This process can be automated using a Bayesian approach, where the model training is sequentially done for a number of hyperparameter optimisation iterations.

Now, we are ready to do a model training using hyperparameter optimisation. - `model_class`: We pick `DirectPred` (a fully connected network) for now. - `config_name`: We use the default/built-in hyperparameter search space for `DirectPred` class. - `target_variables`: 'STUDY' variable contains the type of disease - `n_iter`: We do 5 iterations of hyperparameter optimisation. For demonstration purposes, we set it to a small number. - `plot_losses`: We want to visualize how the training progresses. - `early_stop_patience`: If a training does not show any signs of improving the performance on the `validation` part of the `train_dataset` for at least 10 epochs, we stop the training. This not only significantly decreases the amount spent on training by avoiding unnecessary continuation of unpromising training runs, but also helps avoid over-fitting the network on the training data.

Note 1: Notice how the hyperparameters using in different HPO steps change at each iteration.

Note 2: Also notice that we are running the model for more epochs (500 by default) however, by using “`early_stop_patience=10`”, we avoid lengthy training when validation performance is not improving.

Note 3: Try to follow the the loss curves and the used hyperparameters. See if you can spot which combination yields the lowest/best loss values.

Warning!!: In reality we need to set `n_iter` to higher values so that the optimizer can collect enough data points to learn trends in the parameter space.

```
[25]: # Define a tuner; See n_iter is the number of
tuner = flexynesis.HyperparameterTuning(train_dataset,
                                         model_class = flexynesis.DirectPred,
                                         config_name = "DirectPred",
                                         target_variables = ['STUDY'],
                                         n_iter=5,
                                         plot_losses=True,
                                         early_stop_patience=10)

### Perform Training
model, best_params = tuner.perform_tuning()
```



Validation: | 0/? [00:00<?, ?it/s]

Validate metric

DataLoader 0

STUDY
val_loss

0.32920464873313904
0.32920464873313904

Tuning Progress: 100%| 5/5 [01:05<00:00, 13.10s/it, Iteration=5, Best Loss=0.329]

[INFO] current best val loss: 0.32920464873313904; best params: {'latent_dim': 43, 'hidden_dim_factor': 0.3331314017276735, 'lr': 0.00036705896702579155, 'supervisor_hidden_dim': 10, 'epochs': 500, 'batch_size': 128} since 0 hpo iterations

[26]: *## See which hyperparameter combination was the best best_params*

[26]: {'latent_dim': 43,
'hidden_dim_factor': 0.3331314017276735,
'lr': 0.00036705896702579155,
'supervisor_hidden_dim': 10,
'epochs': 70,
'batch_size': 128}

[27]: *## Evaluate the model and visualising the results*

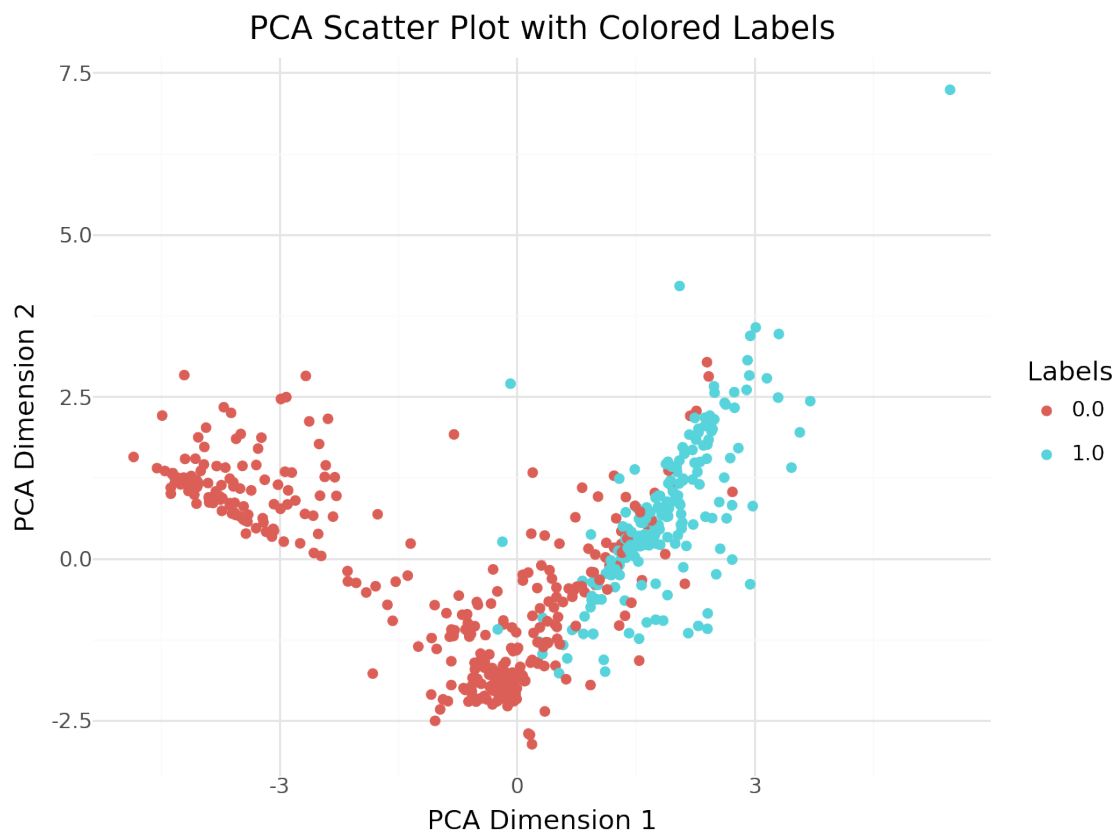
```
flexynesis.evaluate_wrapper(method = 'DirectPred', y_pred_dict=model.
    ↪predict(test_dataset), dataset = test_dataset)
```

```
[27]:
```

	method	var	variable_type	metric	value
0	DirectPred	STUDY	categorical	balanced_acc	0.779739
1	DirectPred	STUDY	categorical	f1_score	0.780883
2	DirectPred	STUDY	categorical	kappa	0.535670
3	DirectPred	STUDY	categorical	average_auroc	0.861822
4	DirectPred	STUDY	categorical	average_aupr	0.764873

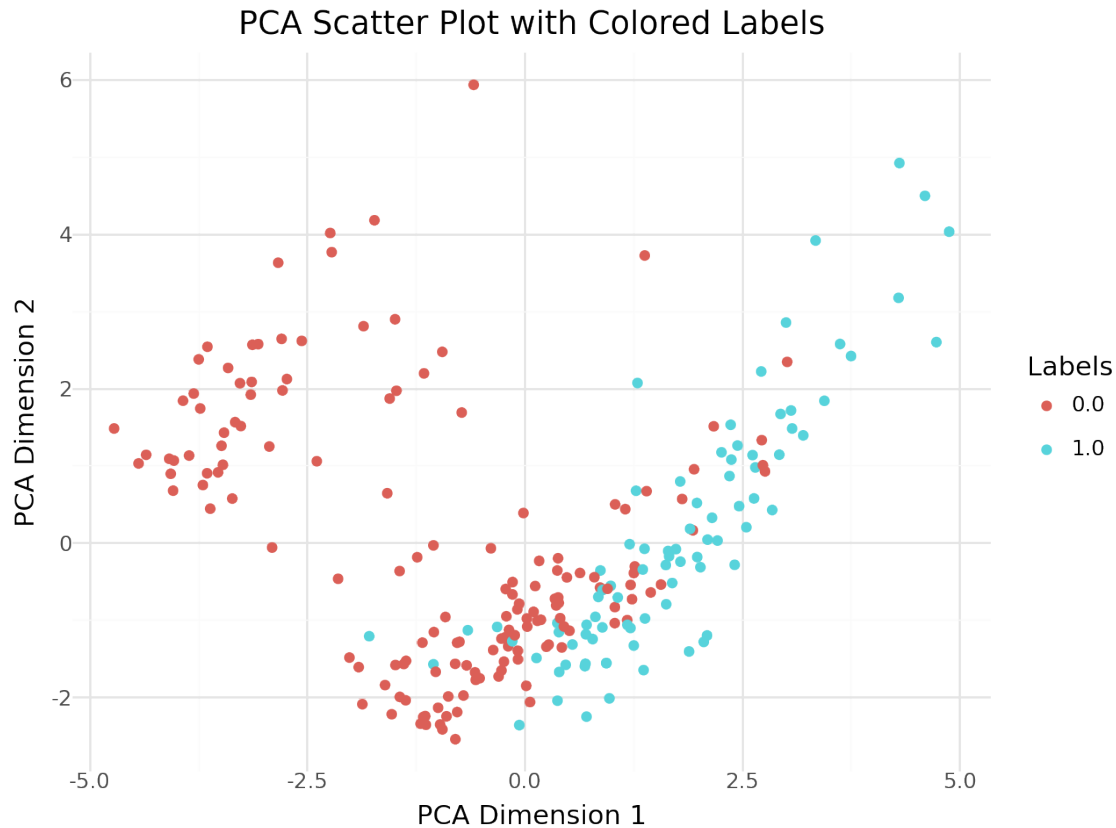
Let's extract the sample embeddings and make a PCA plot and color by the target variable

```
[28]: train_embeddings = model.transform(train_dataset)
flexynesis.plot_dim_reduced(train_embeddings, train_dataset.ann['STUDY'])
```



Repeat the same for the test dataset: extract sample embeddings for test dataset samples and make a PCA plot, colored by “STUDY” variable

```
[29]: test_embeddings = model.transform(test_dataset)
flexynesis.plot_dim_reduced(test_embeddings, test_dataset.ann['STUDY'])
```



1.9 3.1 Exercises

Exercise 1:

Look up what Harrell's C-index means and write down a simple description of what it measures.

Harrell's C-index provides a single value that reflects the effectiveness of a risk prediction model (particularly in survival analysis) in distinguishing between different levels of risk among individuals.

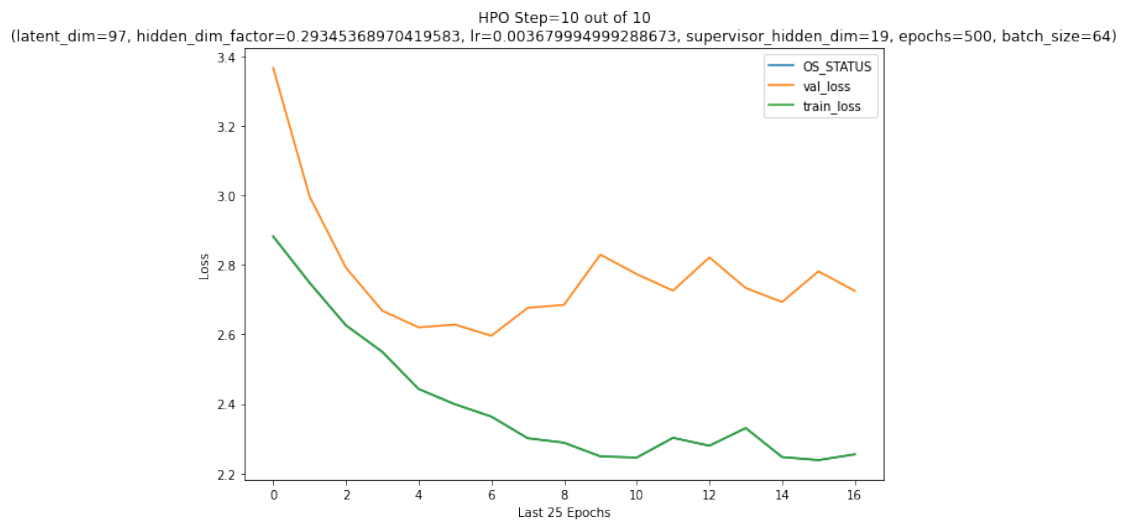
It measures how well a model's predicted risk scores correspond to the actual order of observed event times. A higher C-index indicates that the model more accurately ranks individuals according to their risk of experiencing the event. Exercise 2:

Now, you build a model using hyperparameter tuning (run at least 10 HPO steps) to predict the survival outcomes of patients. Evaluate the final model on test dataset, which computes the "C-index".

Feel free to cheat from the tutorial available here: https://github.com/BIMSBbioinfo/flexynesis/blob/main/example_tutorial.ipynb. See how "OS_STATUS" and "OS_MONTHS" were used.

```
[31]: tuner = flexynesis.HyperparameterTuning(train_dataset,
                                             model_class = flexynesis.DirectPred,
                                             config_name = "DirectPred",
                                             surv_event_var="OS_STATUS",
                                             surv_time_var="OS_MONTHS",
                                             target_variables = [],
                                             n_iter= 10,
                                             plot_losses=True,
                                             early_stop_patience=10)
```

```
[32]: model, best_params = tuner.perform_tuning()
      best_params
```



Validation: | | 0/? [00:00<?, ?it/s]

Validate metric

DataLoader 0

OS_STATUS

2.725167475167475

val_loss

2.725167475167475

Tuning Progress: 100%| | 10/10 [01:12<00:00, 7.22s/it, Iteration=10,
Best Loss=1.86]

[INFO] current best val loss: 1.8555117617617616; best params: {'latent_dim':
85, 'hidden_dim_factor': 0.39904710353861306, 'lr': 0.0047052491639604285,
'supervisor_hidden_dim': 13, 'epochs': 500, 'batch_size': 32} since 6 hpo
iterations


```
[32]: {'latent_dim': 85,
      'hidden_dim_factor': 0.39904710353861306,
      'lr': 0.0047052491639604285,
      'supervisor_hidden_dim': 13,
      'epochs': 17,
      'batch_size': 32}
```

```
[33]: flexynesis.evaluate_wrapper(method = 'DirectPred', y_pred_dict=model.
      ↪predict(test_dataset), dataset = test_dataset,
                                     surv_event_var=model.surv_event_var,
      ↪surv_time_var=model.surv_time_var)
```

```
[33]:      method      var variable_type  metric      value
0  DirectPred  OS_STATUS      numerical  cindex  0.741614
```

Exercise 3:

Again build a model using hyperparameter tuning to predict survival outcomes (as in Exercise 1), however, this time use additional clinical variables as targets.

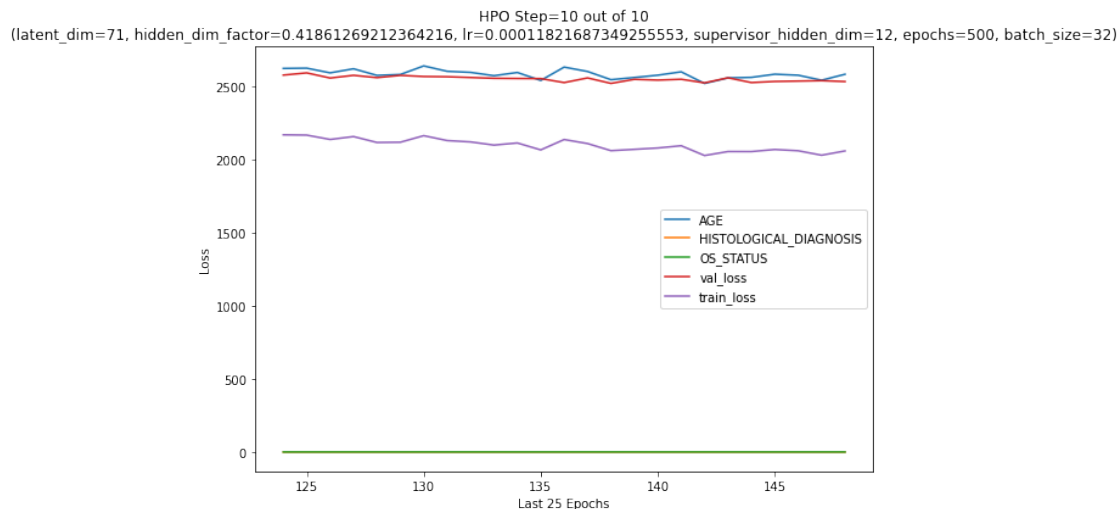
```
flexynesis.HyperparameterTuning(train_dataset,
                                model_class = flexynesis.DirectPred,
                                config_name = "DirectPred",
                                surv_event_var="OS_STATUS",
                                surv_time_var="OS_MONTHS",
                                target_variables = [], => What other variables can you use here
                                ...
```

See if you can get a better C-index using additional target variables.

```
[34]: tuner = flexynesis.HyperparameterTuning(train_dataset,
                                              model_class = flexynesis.DirectPred,
                                              config_name = "DirectPred",
                                              surv_event_var="OS_STATUS",
                                              surv_time_var="OS_MONTHS",
                                              target_variables=["AGE",
      ↪"HISTOLOGICAL_DIAGNOSIS"],
                                              n_iter=10,
                                              plot_losses=True,
                                              early_stop_patience=10)

model, best_params = tuner.perform_tuning()

flexynesis.evaluate_wrapper(method = 'DirectPred',
                             y_pred_dict=model.predict(test_dataset),
                             dataset = test_dataset)
```



Validation: | 0/? [00:00<?, ?it/s]

Validate metric	DataLoader 0
AGE	2529.7265625
HISTOLOGICAL_DIAGNOSIS	0.9858999252319336
OS_STATUS	1.9452204608454609
val_loss	2532.657767969291

Tuning Progress: 100%| 10/10 [08:01<00:00, 48.18s/it, Iteration=10, Best Loss=199]

[INFO] current best val loss: 198.84827784132898; best params: {'latent_dim': 58, 'hidden_dim_factor': 0.21785352913840994, 'lr': 0.0020043902744191506, 'supervisor_hidden_dim': 15, 'epochs': 500, 'batch_size': 64} since 7 hpo iterations

[34]:

	method	var	variable_type	metric \
0	DirectPred	AGE	numerical	mse
1	DirectPred	AGE	numerical	r2
2	DirectPred	AGE	numerical	pearson_corr
3	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	balanced_acc
4	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	f1_score
5	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	kappa
6	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	average_auroc
7	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	average_aupr
8	DirectPred	OS_STATUS	numerical	mse

9	DirectPred	OS_STATUS	numerical	r2
10	DirectPred	OS_STATUS	numerical	pearson_corr

	value
0	207.982624
1	0.229201
2	0.478750
3	0.500114
4	0.543156
5	0.387137
6	NaN
7	NaN
8	3.985677
9	0.197059
10	0.443913

```
[ ]:
```

1.10 3.2 Survival-risk subtypes

Use the best model from the above exercises to inspect sample embeddings categorized by survival risk scores.

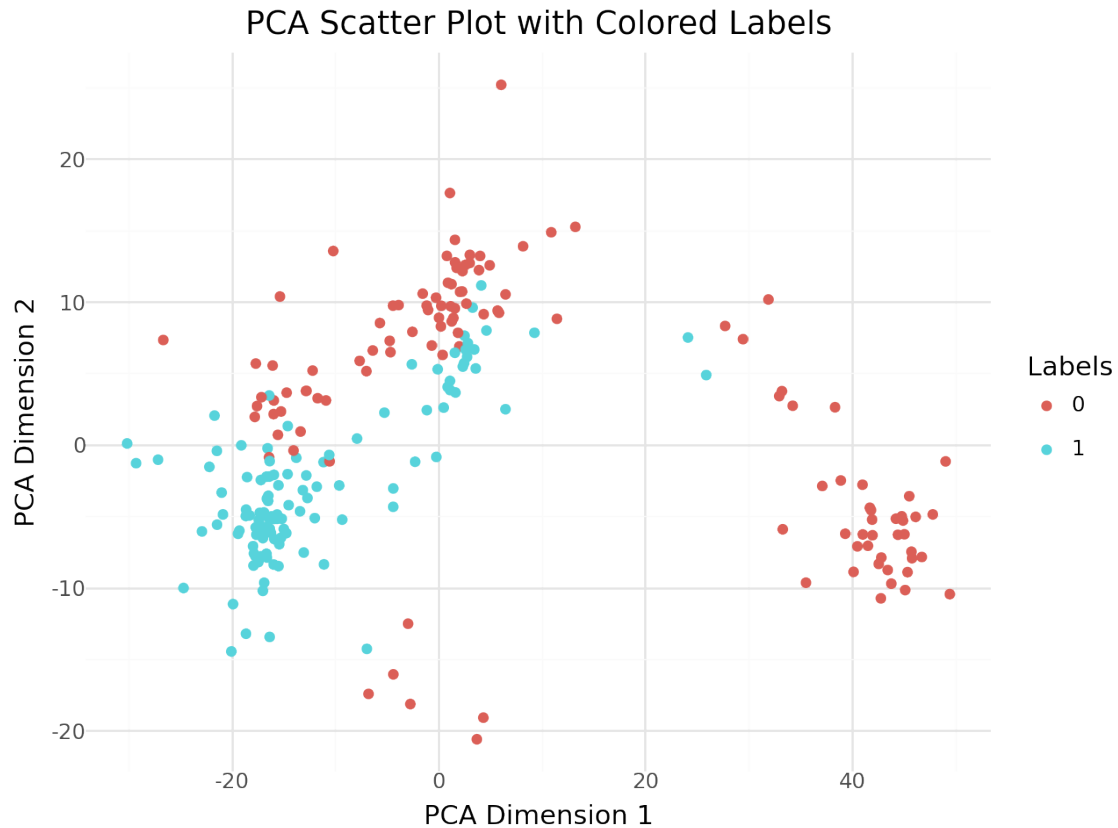
Let's group the samples by predicted survival risk scores into 2 groups and visualize the sample embeddings colored by risk subtypes.

Notice: You can use the code-below to get survival risk groups, however, notice that you must have built a model with "OS_STATUS" already.

```
[35]: # get model outputs for survival variable
outputs = model.predict(test_dataset)['OS_STATUS'].flatten()
risk_scores = np.exp(outputs)
# Define quantile thresholds
quantiles = np.quantile(risk_scores, [0.5])
# Assign groups based on quantiles
groups = np.digitize(risk_scores, quantiles)
```

```
[36]: # Extract sample embeddings
E = model.transform(test_dataset)
```

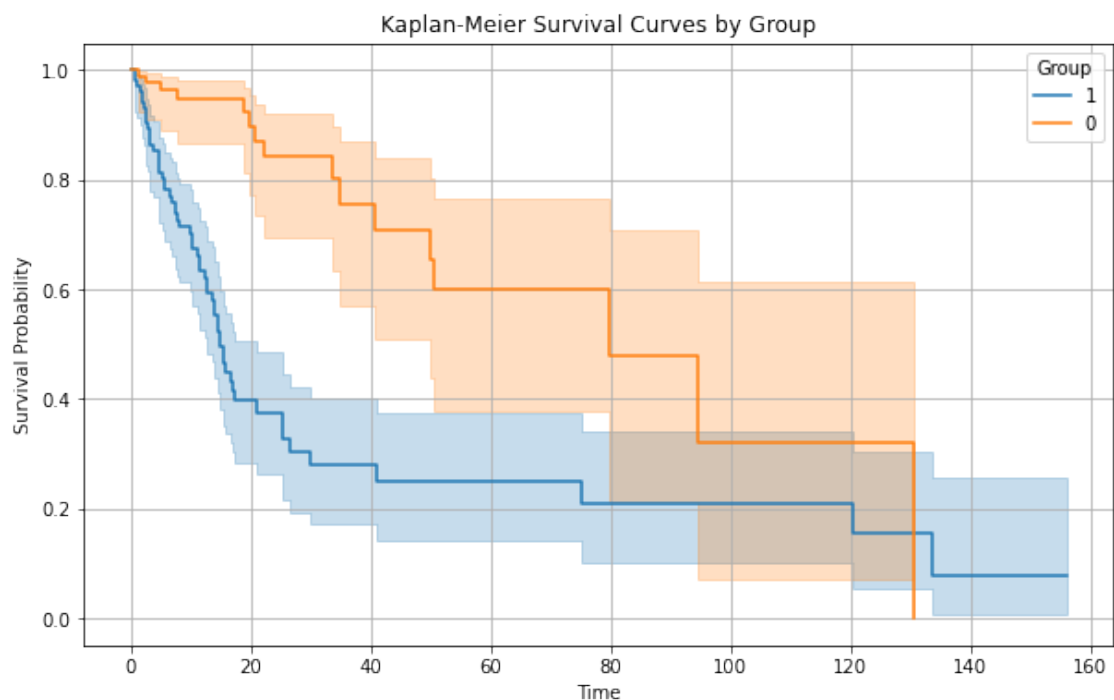
```
[38]: flexynesis.plot_dim_reduced(E, groups)
```



Let's also see the Kaplan Meier Curves of the risk subtypes

```
[41]: # remove samples with NA values first
durations = test_dataset.ann['OS_MONTHS']
events = test_dataset.ann['OS_STATUS']
valid_indices = ~torch.isnan(durations) & ~torch.isnan(events)
```

```
[42]: flexynesis.plot_kaplan_meier_curves(durations[valid_indices],
    ↪ events[valid_indices], groups[valid_indices])
```



1.10.1 Finding survival-associated markers

We can also compute feature importance scores for prediction of overall survival.

```
[43]: model.compute_feature_importance(train_dataset, 'OS_STATUS')
```

```
[44]: # get top 10 features
flexynesis.get_important_features(model, var = 'OS_STATUS', top=10)
```

```
[44]:  target_variable  target_class  target_class_label  layer  name  importance
0      OS_STATUS           0                mut    IDH1    0.676176
1      OS_STATUS           0                mut    ATRX    0.444618
2      OS_STATUS           0                mut    TP53    0.349603
3      OS_STATUS           0                mut    IDH2    0.078091
4      OS_STATUS           0                mut    PIK3CA   0.051682
5      OS_STATUS           0                mut    ARID1A   0.039856
6      OS_STATUS           0                mut    MUC16    0.037684
7      OS_STATUS           0                mut    COL6A3   0.031301
8      OS_STATUS           0                cna    MIR603   0.030820
9      OS_STATUS           0                mut    BRD3     0.027779
```

1.10.2 Comparing top markers with clinical covariates

Let's build a linear Cox-PH model including the top 5 markers and other clinical variables such as histological diagnosis, disease type (STUDY), age, and sex.

```
[45]: # define a data.frame with clinical covariates and top markers along with
      ↪survival endpoints
vars = ['AGE', 'SEX', 'HISTOLOGICAL_DIAGNOSIS', 'STUDY', 'OS_MONTHS',
      ↪'OS_STATUS']
# read clinical variables
df_clin = pd.concat(
    [pd.DataFrame({x: train_dataset.ann[x] for x in vars}, index=train_dataset.
    ↪samples),
    pd.DataFrame({x: test_dataset.ann[x] for x in vars}, index=test_dataset.
    ↪samples)],
    axis = 0)
# get top 5 survival markers and extract the input data for these markers for
      ↪both training and test data
imp = flexynesis.get_important_features(model, var = 'OS_STATUS', top=5)
df_imp = pd.concat([train_dataset.get_feature_subset(imp), test_dataset.
    ↪get_feature_subset(imp)], axis=0)

# combine markers with clinical variables
df = pd.concat([df_imp, df_clin], axis = 1)
# remove samples without survival endpoints
df = df[df['OS_STATUS'].notna()]
df
```

```
[45]:
```

	mut_IDH1	mut_ATRX	mut_TP53	mut_IDH2	mut_PIK3CA	AGE	SEX	\
TCGA-TM-A84H	0.982173	1.707482	1.235829	-0.148522	-0.307389	44.0	0.0	
TCGA-06-0650	-1.018150	-0.585658	-0.809174	-0.148522	-0.307389	39.0	0.0	
TCGA-E1-A7Y0	0.982173	-0.585658	-0.809174	-0.148522	-0.307389	45.0	1.0	
TCGA-DU-7290	-1.018150	-0.585658	-0.809174	-0.148522	-0.307389	45.0	0.0	
TCGA-06-0168	-1.018150	-0.585658	-0.809174	-0.148522	-0.307389	59.0	0.0	
...	...	...	...	...	...	...	...	
TCGA-DB-A750	0.982173	-0.585658	1.235829	-0.148522	-0.307389	29.0	1.0	
TCGA-FG-A70Z	-1.018150	-0.585658	-0.809174	-0.148522	-0.307389	53.0	0.0	
TCGA-DU-6394	0.982173	-0.585658	-0.809174	-0.148522	-0.307389	53.0	1.0	
TCGA-HT-8011	-1.018150	-0.585658	-0.809174	-0.148522	-0.307389	55.0	1.0	
TCGA-DH-A7UR	-1.018150	-0.585658	-0.809174	6.733003	-0.307389	59.0	0.0	

	HISTOLOGICAL_DIAGNOSIS	STUDY	OS_MONTHS	OS_STATUS
TCGA-TM-A84H	2.0	0.0	24.2	0.0
TCGA-06-0650	1.0	1.0	23.6	1.0
TCGA-E1-A7Y0	3.0	0.0	75.0	1.0
TCGA-DU-7290	0.0	0.0	10.3	1.0
TCGA-06-0168	1.0	1.0	19.6	1.0
...	...	...	...	...
TCGA-DB-A750	0.0	0.0	30.7	0.0
TCGA-FG-A70Z	2.0	0.0	10.8	0.0
TCGA-DU-6394	3.0	0.0	22.4	1.0
TCGA-HT-8011	0.0	0.0	16.2	0.0

TCGA-DH-A7UR	3.0	0.0	54.2	0.0
--------------	-----	-----	------	-----

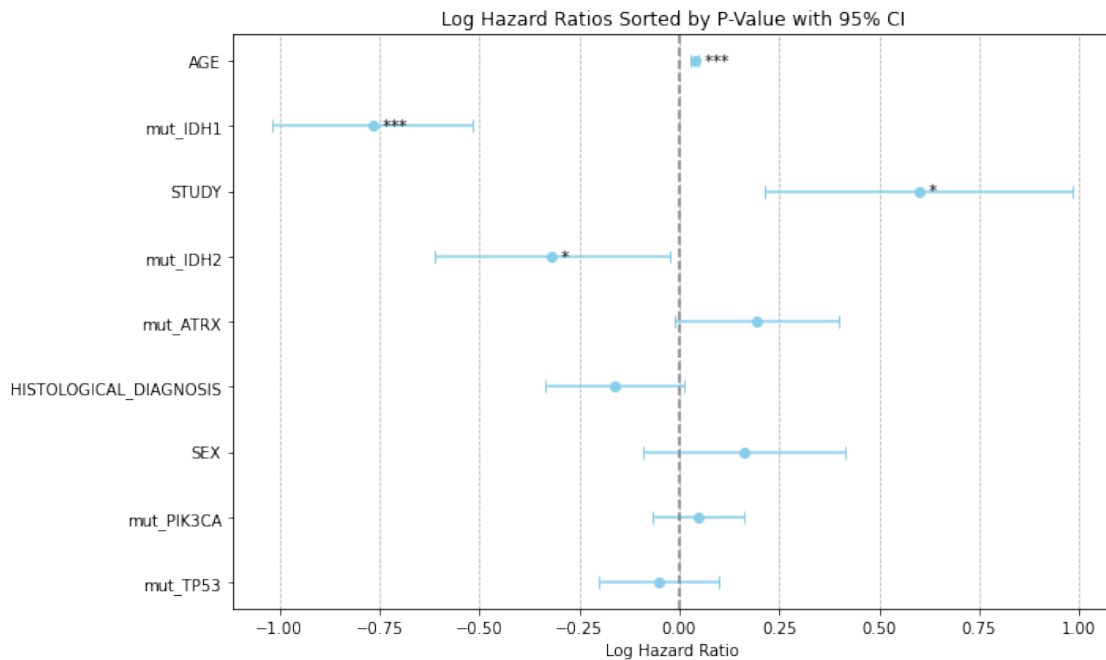
[729 rows x 11 columns]

```
[46]: # build a cox model
coxmodel = flexynesis.build_cox_model(df, 'OS_MONTHS', 'OS_STATUS')
```

No low variance features were removed based on event conditioning.

```
[47]: # visualize log-hazard ratios sorted by p-values
flexynesis.plot_hazard_ratios(coxmodel)
```

/gnu/store/88qdfdp9h0003w4wp1rv46kxc3dpp8j6-profile/lib/python3.10/site-packages/flexynesis/utis.py:764: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`



1.11 3.3 Final Exercise

- Inspect the top 10 markers from section 3.2 and see if they have been characterized in the literature as important markers for Glioma disease progression.

```
[49]: flexynesis.get_important_features(model, var = 'OS_STATUS', top=10)
```

```
[49]: target_variable target_class target_class_label layer name importance
0 OS_STATUS 0 mut IDH1 0.676176
```

1	OS_STATUS	0	mut	ATRX	0.444618
2	OS_STATUS	0	mut	TP53	0.349603
3	OS_STATUS	0	mut	IDH2	0.078091
4	OS_STATUS	0	mut	PIK3CA	0.051682
5	OS_STATUS	0	mut	ARID1A	0.039856
6	OS_STATUS	0	mut	MUC16	0.037684
7	OS_STATUS	0	mut	COL6A3	0.031301
8	OS_STATUS	0	cna	MIR603	0.030820
9	OS_STATUS	0	mut	BRD3	0.027779

- 1.11.1 IDH1 [Turcan, Sevin, et al. “IDH1 mutation is sufficient to establish the glioma hypermethylator phenotype.” *Nature* 483.7390 (2012): 479-483.]
- 1.11.2 ATRX [Haase, Santiago, et al. “Mutant ATRX: uncovering a new therapeutic target for glioma.” *Expert opinion on therapeutic targets* 22.7 (2018): 599-613.]
- 1.11.3 TP53 [Noor, Humaira, et al. “TP53 mutation is a prognostic factor in lower grade glioma and may influence chemotherapy efficacy.” *Cancers* 13.21 (2021): 5362.]
- 1.11.4 IDH2 [Cohen, Adam L., Sheri L. Holmen, and Howard Colman. “IDH1 and IDH2 mutations in gliomas.” *Current neurology and neuroscience reports* 13 (2013): 1-7.]
- 1.11.5 PIK3CA [Hartmann, Christian, et al. “PIK3CA mutations in glioblastoma multiforme.” *Acta neuropathologica* 109 (2005): 639-642.]
- 1.11.6 ARID1A [Xiao, Menglin, et al. “Deep-targeted gene sequencing reveals ARID1A mutation as an important driver of glioblastoma.” *CNS Neuroscience & Therapeutics* 30.4 (2024): e14698.]
- 1.11.7 MUC16 [Erickson, Anna, et al. “Mucins as Precision Biomarkers in Glioma: Emerging Evidence for Their Potential in Biospecimen Analysis and Outcome Prediction.” *Biomedicines* 12.12 (2024): 2806.]
- 1.11.8 COL6A3 [Turtoi, Andrei, et al. “Accessibilome of human glioblastoma: collagen-VI-alpha-1 is a new target and a marker of poor outcome.” *Journal of proteome research* 13.12 (2014): 5660-5669.]
- 1.11.9 MIR603 [Guo, Mian, et al. “miR-603 promotes glioma cell growth via Wnt/ -catenin pathway by inhibiting WIF1 and CTNNBIP1.” *Cancer letters* 360.1 (2015): 76-86.]
- 1.11.10 BRD3 [Yang, Hua, et al. “BRD4: An emerging prospective therapeutic target in glioma.” *Molecular Therapy-Oncolytics* 21 (2021): 1-14.]

[]: